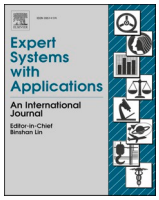
Expert Systems With Applications 274 (2025) 127012



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





Elite neighborhood search and dynamic feature probability-guided crow search algorithm for high-dimensional feature selection

Shaobo Deng *, Hangyu Liu, Weihu Zhu, Kexing Li, Min Li

School of Information Engineering, Nanchang Institute of Technology, No. 289 Tianxiang Road, Nanchang, Jiangxi, PR China

ARTICLE INFO

Keywords:
Feature selection
Crow search algorithm (CSA)
Time varying dynamic awareness probability
Elite neighborhood search
Feature probability

ABSTRACT

Crow search algorithm (CSA) is a metaheuristic algorithm inspired by the collective behavior of crows. It has been applied in various fields because of its fewer parameters and easy implementation. However, it has some drawbacks: (1) It is difficult to balance global search and local search, which leads to premature convergence. (2) Local search and global search are too random, leading to the possibility of more invalid solutions. To address such problems, this paper proposes the Elite Neighborhood Search and Dynamic Feature Probability-Guided Crow Search Algorithm (NNS-DFPG-CSA) for feature selection. To address the difficulty of balancing global search and local search, we propose a dynamic awareness probability based on time-varying to balance global search and local search. To address the randomness of local search, we use the elite neighborhood search strategy to guide the local search of crows. To address the randomness and low search efficiency of global search, we use the feature probability to guide the global search of the algorithm. During the experiments, this paper uses 16 high-dimensional datasets and three evaluation metrics to evaluate NNS-DFPG-CSA. Experimental results show that NNS-DFPG-CSA has satisfactory classification accuracy and feature subset length.

1. Introduction

In the era of big data, datasets in diverse domains exhibit exponential growth in both scale and complexity. Consequently, effective extraction of valuable information from raw datasets has emerged as a formidable challenge. A pivotal role in addressing this challenge is played by feature selection, a critical component of data preprocessing (Seijo-Pardo et al., 2018; Dash, 2018). It involves selecting a subset of the most informative features from the original dataset, reducing computational costs, and improving the performance of machine learning algorithms, thus enabling more accurate and efficient data analysis (M. Dash & Liu, 1997).

Feature selection methods are mainly categorized into three groups: Filter, Wrapper, and Hybrid algorithms (GuyonIsabelle & ElisseeffAndré, 2003; Hancer et al., 2017; Li et al., 2013; Deng et al., 2022; Ke et al., 2022). Filter-based methods seek feature subsets based on specific feature characteristics such as similarity and relevance (M. Dash & Liu, 2003; Demšar, 2006; Kira & Rendell, 1992; Cover & Thomas, 2005). In contrast, wrapper-based methods leverage classifier models to assess feature subsets through cross-validation, ultimately determining the performance of candidate feature subsets. Consequently, wrapper-based methods incur higher computational costs. Conversely, filter-based methods do not necessitate a classifier for evaluation, which leads to reduced costs and expedited execution but often at the expense of reduced accuracy.

Recognizing the constraints of both filter and wrapper methods, Lu et al. (2017) introduced a hybrid approach that combines the advantages of both methodologies. The hybrid method comprises two sequential steps. Initially, a filter method is applied to eliminate the majority of redundant and irrelevant features, leaving behind only the pivotal ones. Subsequently, a wrapper-based method is employed to select the optimal feature subset from the retained features (Das et al., 2017). This hybrid methodology effectively harnesses the advantages of both filter and wrapper methods, resulting in superior classification accuracy and simultaneously reducing time costs.

Although numerous feature selection (FS) techniques have been proposed over the past two decades, the problem remains inherently challenging. For an N-dimensional dataset, the number of possible feature subsets is $2^N - 1$, making FS an NP-hard combinatorial problem (Unler et al., 2010). Both exhaustive and greedy search methods face

* Corresponding author.
E-mail addresses: houjiyuan@nit.edu.cn (S. Deng), hangyuliu@nit.edu.cn (H. Liu), weihuzhu@nit.edu.cn (W. Zhu), kexingli@nit.edu.cn (K. Li), liminghuadi@nit.edu.cn (M. Li).

https://doi.org/10.1016/j.eswa.2025.127012
Received 8 August 2024; Received in revised form 6 December 2024; Accepted 25 January 2025
Available online 24 February 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

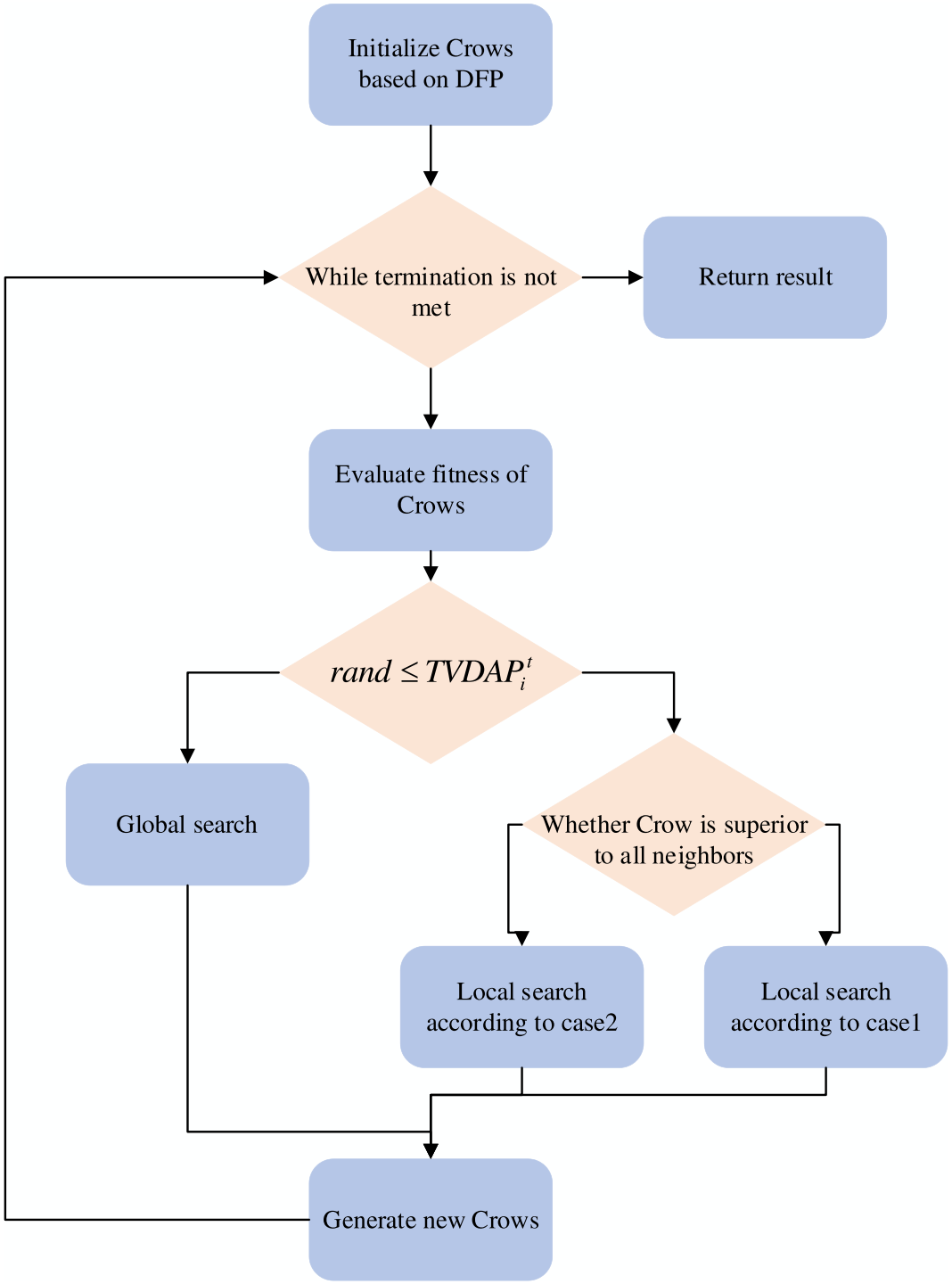


Fig. 1. Flowchart of NNS-DFPG-CSA.

significant computational costs and are unable to identify the optimal subset in a reasonable timeframe (Dong et al., 1999; Faris et al., 2018). While random search can alleviate some of these challenges, it can still revert to exhaustive search in the worst case, evaluating all possible feature combinations (Taradeh et al., 2019).

Recently, metaheuristic optimization algorithms have gained increasing attention in FS due to their high computational efficiency and strong global search capabilities (Hancer, Xue, Zhang, et al., 2017). Multiple algorithms have been developed based on this approach. Faris et al. (2019) proposed a novel approach called the time-varying binary Salp Swarm Algorithm (TVBSSA) for feature selection. This algorithm introduces a dynamic mechanism to adaptively change the number of leaders, enabling the algorithm to continuously adjust its exploration capability. Sindhu et al. (2017) proposed an improved version of the Sine Cosine Algorithm (ISCA), which incorporates an elite strategy and a novel best solution updating mechanism. This modified algorithm aims to select the optimal features/attributes and improve the classification accuracy of feature selection. Mohammad Tubishat et al. (2019) proposed an improved salp swarm algorithm. This algorithm's main contribution lies in improving the population diversity during the initialization phase and introducing a new local search algorithm. Abdel-Basset et al. (2019) proposed a grey wolf optimization algorithm

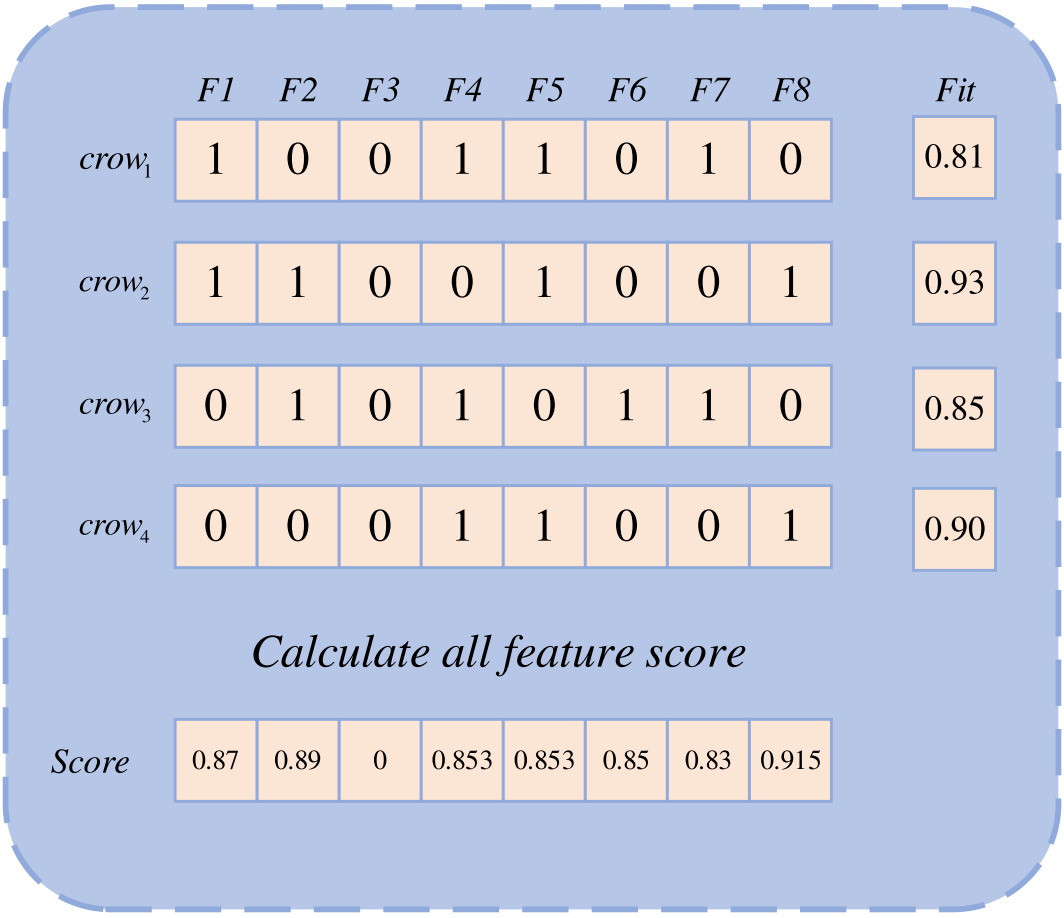


Fig. 2. Calculate all feature score.

(TMGWO) that combines two-stage mutation to address the feature selection problem. Of course, there are also some classic algorithms, such as Genetic Algorithm (GA) (Holland, 1975; S. Deng et al., 2023), Particle Swarm Optimization (PSO) (Tran et al., 2016; Mafarja & Sabar, 2018) and Differential Evolution (DE) (Moradi & Gholampour, 2016; Storn & Price, 1997; Chattopadhyay et al., 2016; Gong & Cai, 2013).

The Crow Search Algorithm (CSA) is an emerging optimization algorithm, which was initially introduced by Askarzadeh (2016). This algorithm addresses optimization problems by simulating the behavior of crows hiding food. CSA's evaluation results indicate that it exhibits exceptional performance in optimization problems, particularly those posing challenges in scientific and engineering domains. Additionally, the algorithm offers the advantages of straightforward implementation and requiring only a small number of parameters. It can be seen that CSA is a promising technology. However, it also presents challenges, such as the difficulty in fine-tuning the AP parameter to balance local and global search, which can result in convergence to local optima. Additionally, CSA relies on a stochastic approach for global search, which may affect optimization efficiency due to the high level of randomness.

Chaudhuri and Sahu (2020) proposed a binary crow search algorithm with time-varying flight lengths (BCSA-TVFL). This algorithm enhances CSA by replacing the fixed crow flight lengths, and allows the crows' flight lengths to continuously change during the algorithm's iterations "while" searching. Ouadfel and Elaziz (2020) introduced an enhanced version of CSA called ECSA for searching for the optimal feature subset. The algorithm proposes a dynamic perceptual probability to further enhance the exploration capability of the algorithm. It also uses dynamic local neighborhoods to guide the local search instead of crows following randomly.

As mentioned above, CSA relies on two key operations: exploration and utilization. Exploration involves searching for new regions in the solution space, while exploitation focuses on refining the solution near previously visited regions. To obtain optimal results, algorithms must strike a balance between exploration and exploitation. Many researchers believe that a proper balance between these two operations is critical to the effectiveness of an algorithm. However, an improper balance often leads the algorithm to a local optimum. As the results in Section 4.4.1 show, existing CSA and its improved algorithms are unable to effectively balance exploration and exploitation while improving search capabilities when faced with high-dimensional feature selection. Therefore, the motivation for the research in this paper is to propose an improvement program to address this problem.

In this paper, we put forth a novel high-dimensional feature selection method, the Elite Neighborhood Search and Dynamic Feature Probability-Guided Crow Search Algorithm (NNS-DFPG-CSA), as a potential solution to the aforementioned challenges. The NNS-DFPG-CSA is a hybrid feature selection method, comprises two stages. Initially, the mutual information is employed to filter out a significant portion of features, retaining the top k % of features. Subsequently, in the second stage, NNS-DFPG-CSA utilizes CSA to further evaluate the feature subset after mutual information filtering. The crow positions generated by NNS-DFPG-CSA use binary encoding, as opposed to the floating-point encoding in CSA. This enables the algorithm to directly assess the fitness of each crow position without the need for converting floating-point numbers to binary, thus that saves computational costs. The main contributions of this article are shown as follows:

(1) This paper proposes a method that uses dynamic feature probabilities to guide the initialization of the crow's position.

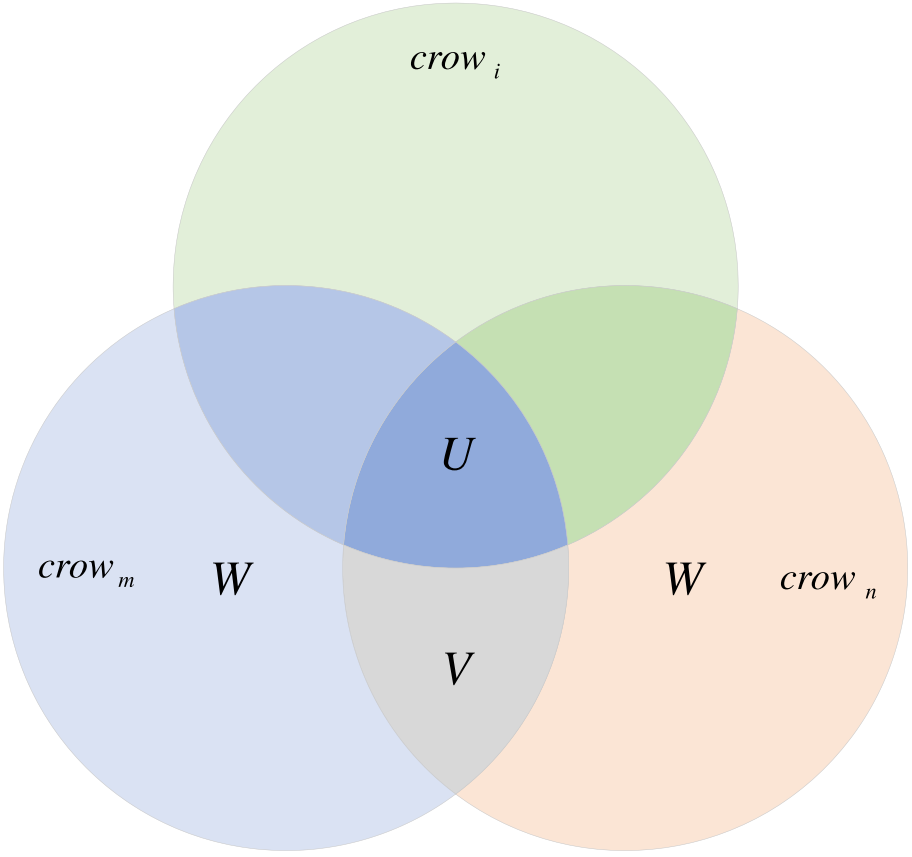


Fig. 3. Simplified schematic of case(2) strategy.

Table 1
Descriptive summary of microarray datasets.

Datasets	No. of total features	No. of samples	No. of classes
ALLAML	7129	72	2
Arcene	10,000	200	2
BreastCancer1	24,482	96	2
BreastCancer2	22,283	104	2
CNS	7130	59	2
CrohnDisease	22,283	127	3
GLI	22,283	85	2
Leukemia	7129	72	2
Lung	12,601	202	5
LungCancer	12,533	181	2
MLL	12,583	71	3
Orlraws10P	10,304	100	10
Ovarian	15,154	253	2
Prostate	12,600	102	2
SMKCAN	19,993	187	2
TOX_171	6748	171	4

(2) This paper proposes a novel approach for updating dynamic feature probabilities through a new feature scoring strategy.
(3) This paper improves the CSA's local search. We introduce a new similarity-based elite neighborhood search mechanism, which allow crows to select another crow from their neighbors to follow during local search. So that results in higher local search efficiency and reduces generation probability of suboptimal solutions.
(4) We strengthen CSA's global search capabilities. Classical CSA's global search essentially involves randomly generating a new crow position in the search space. This paper uses dynamic feature probabilities to guide the generation of new crow positions, and then reduce the production of invalid solutions.

Table 2
The parameter settings.

Algorithms	Parameters	Values
DE	Crossover probability CR	0.9
	Scaling factor F	0.5
GA	Crossover probability CR	0.8
	Mutation rate mu	1/D
PSO	Inertia factor ω	0.9
	c_1	2
	c_2	2
TVBSSA	maximum number of hidden neurons	1024
ISCA	Number of elites	2
ISSA	maximum iteration for local search LSA	10
TMGWO	A	[0,2]
	M_p	0.5
CSA	Awareness Probability AP	0.1
ECSA	AP_{min}	0.1
	AP_{max}	0.8
BCSA-TVFL	Awareness Probability AP	0.1
NNS-DFPG-CSA	AP^t_{min}	0.1
	AP^t_{max}	0.8
	The rate of change of feature probabilities ξ	$5/T$

The remaining sections of this article are shown as follows: section 2 presents related work, and primarily focuses on crow search algorithms and the first stage filtering Algorithm based on mutual information; Section 3 introduces the Elite Neighborhood Search and Dynamic Feature Probability-Guided Crow Search Algorithm. Section 4 introduces the numerical results of the algorithm and compares the performance of various algorithms, and section 5 provides a summary of the research.

Table 3
Comparison of NNS-DFPG-CSA and CSA based methods in terms of average and std of accuracy.

Datasets	Stat. measure	NNS-DFPG-CSA	CSA based		
			CSA	ECSA	BCSA-TVFL
ALLAML	mean	**0.8627**	0.8088 (+)	0.7936 (+)	0.7977 (+)
	std	0.0109	0.0117	0.0106	0.0083
Arcene	mean	**0.9178**	0.7918 (+)	0.7967 (+)	0.7921 (+)
	std	0.0140	0.0082	0.0032	0.0060
BreastCancer1	mean	**0.7716**	0.6611 (+)	0.6716 (+)	0.6200 (+)
	std	0.0189	0.0103	0.0155	0.0074
BreastCancer2	mean	**0.9610**	0.8149 (+)	0.8692 (+)	0.8272 (+)
	std	0.0101	0.0131	0.0078	0.0129
CNS	mean	**0.7558**	0.6432 (+)	0.6529 (+)	0.6303 (+)
	std	0.0154	0.0113	0.0120	0.0097
CrohnDisease	mean	**0.8486**	0.6626 (+)	0.6393 (+)	0.6393 (+)
	std	0.0239	0.0065	0.0052	0.0052
GLI	mean	**0.9195**	0.8537 (+)	0.8274 (+)	0.7960 (+)
	std	0.0131	0.0070	0.0108	0.0051
Leukemia	mean	**0.8503**	0.7604 (+)	0.7708 (+)	0.7872 (+)
	std	0.0151	0.0133	0.0158	0.0049
Lung	mean	**0.8197**	0.7595 (+)	0.7679 (+)	0.7615 (+)
	std	0.0099	0.0033	0.0045	0.0031
LungCancer	mean	**0.9444**	0.9017 (+)	0.9006 (+)	0.8922 (+)
	std	0.0080	0.0025	0.0030	0.0027
MLL	mean	**0.8057**	0.5586 (+)	0.6257 (+)	0.5386 (+)
	std	0.0294	0.0149	0.0254	0.0065
Orlraws10P	mean	**0.8584**	0.7637 (+)	0.7597 (+)	0.7517 (+)
	std	0.0075	0.0046	0.0078	0.0049
Ovarian	mean	**0.8432**	0.7541 (+)	0.7541 (+)	0.7553 (+)
	std	0.0109	0.0053	0.0509	0.0031
Prostate	mean	**0.8340**	0.7020 (+)	0.7028 (+)	0.6848 (+)
	std	0.0165	0.0147	0.0118	0.0085
SMKCAN	mean	**0.8017**	0.6943 (+)	0.7167 (+)	0.7210 (+)
	std	0.0114	0.0068	0.0112	0.0054
TOX_171	mean	**0.8539**	0.6982 (+)	0.6912 (+)	0.6934 (+)
	std	0.0239	0.0070	0.0121	0.0080
Avg. Accuracy		**0.8530**	0.7393	0.7463	0.7305

2. Relate work

2.1. Representative metaheuristic-based FS methods

Over the past few decades, many metaheuristic-based feature selection methods have been developed. This section will review some of the most representative and cutting-edge methods.

Metaheuristic algorithms have been shown to perform better for solving combinatorial problems (Talbi, 2009). These algorithms are capable of exploring a vast number of potential feature subsets, allowing them to identify near-optimal solutions within a reasonable timeframe. Song et al. (2020) introduced a novel feature selection approach based on bare-bones Particle Swarm Optimization (BBPSO) combined with mutual information. By developing an efficient swarm initialization strategy rooted in label correlation, the algorithm fully leverages the relationship between features and class labels, accelerating convergence and yielding improved feature subsets. Wang et al. (2020) proposed an attribute learning strategy that updates populations associated with features, leading to the development of a bacterial community-based feature selection algorithm aimed at enhancing customer classification accuracy in personalized product recommendations. C. Li et al. (2022) introduced a novel Evolutionary Bacterial Swarm Cuckoo Algorithm (EBSCA) that uses an adaptation-based balancing factor and positive crossover strategy, achieving outstanding results on both unimodal and multimodal benchmark test functions. Kale and Yüzgeç (2021) proposed four enhanced versions of the Salp Cuckoo Algorithm (SCA) to address the problem of SCA becoming trapped in local search regions.

Recently developed metaheuristic algorithms are widely applied across various domains. Chakraborty et al. (2022) proposed a hybrid machine learning classification technique that combines ensemble methods with the Enhanced Gray Wolf Optimization (E-GWO) for feature selection to analyze complex biomedical data. Nadimi-Shahraki et al. (2022) introduced a new multi-trial vector method, MTV, into the Differential Evolution (DE) algorithm, leading to the development of Multi-Trial Vector Based Differential Evolution (MTDE), which effectively solves four complex engineering problems. In (Elaziz et al., 2019), a hybrid feature selection algorithm (OMFODE) based on the Opposition-Based Learning (OBL), Moth Flame Optimization (MFO), and Differential Evolution (DE) was proposed. In OMFODE, OBL initializes solutions, while DE enhances the search capability of MFO. Gupta et al. (2018) presented an optimized version of the Crow Search Algorithm (OCSA), designed to improve the diagnosis of Parkinson's disease, demonstrating superior diagnostic performance.

As outlined above, although numerous metaheuristic-based feature selection methods have been proposed, there remains a need for the development of new techniques. According to the No-Free-Lunch theorem (Wolpert & Macready, 1997), no single algorithm can outperform others across all datasets. Thus, there is still a demand for new feature selection methodologies.

2.2. Redundant feature filtering based on mutual information

As mentioned in the introduction, current feature selection methods are generally categorized into three types: filter algorithms, wrapper algorithms, and hybrid algorithms. Hybrid algorithms effectively combine the strengths of the other two types. Based on this, the NNS-DFPG-CSA proposed in this paper is designed as a hybrid algorithm.

In the first stage of hybrid feature selection methods, some form of filter measure (FM) is usually used to remove irrelevant or weakly relevant features from the dataset (Kononenko, 1994; Peng et al., 2005; López et al., 2021; Salesi et al., 2021). This initial filtering process aims to reduce the number of features, thereby reducing the computational burden of subsequent processing and improving the performance of the model. Many studies (as described in (Yu & Liu, 2003; Song, Zhang, Guo, et al., 2020; Song et al., 2021)) have not only relied on predefined thresholds in the screening process, but have also used the correlation between features and class labels for selection. By calculating the information gain between features and class labels, it is possible to quantify the importance of each feature for the classification task, and thus identify and reject weakly relevant features.

The advantage of this approach is that it is able to capture not only linear but also complex non-linear relationships, and therefore shows good performance when dealing with different types of data. Among these methods, Mutual Information (MI) has received a lot of attention. By adopting the mutual information approach in the first stage, features that have a significant impact on the target variable can be effectively retained, while redundant and noisy features are eliminated, laying the foundation for the subsequent feature selection process.

Mutual information (Viola, 1995) is a concept based on information theory and is a metric used to measure dependencies between variables. As shown in Eq.(1).

Table 4
Comparison of NNS-DFPG-CSA and other methods in terms of average and std of accuracy.

Datasets	Stat.measure	NNS-DFPG-CSA	Other Method						
			GA	DE	PSO	ISCA	ISSA	TVBSSA	TMGWO
ALLAML	mean	0.8627	**0.8751(≈)**	0.8119(+)	0.8240(+)	0.8511(≈)	0.8235(+)	0.7707(+)	0.8125(+)
	std	0.0109	0.0084	0.0148	0.0263	0.0199	0.0058	0.0058	0.0136
Arcene	mean	**0.9178**	0.9116(≈)	0.8487(+)	0.8584(+)	0.8212(+)	0.8297(+)	0.7892(+)	0.8152(+)
	std	0.0140	0.0064	0.0101	0.0260	0.0188	0.0141	0.0067	0.0066
BreastCancer1	mean	0.7716	**0.7884(≈)**	0.7063(+)	0.7189(+)	0.7411(+)	0.6747(+)	0.6316(+)	0.6568(+)
	std	0.0189	0.0179	0.0120	0.0236	0.0368	0.0173	0.0115	0.0084
BreastCancer2	mean	**0.9610**	0.9584(≈)	0.9070(+)	0.9220(+)	0.9051(+)	0.8954(+)	0.8318(+)	0.8330(+)
	std	0.0101	0.0124	0.0124	0.0212	0.0255	0.0131	0.0122	0.0073
CNS	mean	0.7558	**0.7855(−)**	0.7091(+)	0.7409(≈)	0.7485(≈)	0.7170(+)	0.6527(+)	0.6467(+)
	std	0.0154	0.0163	0.0192	0.0216	0.0249	0.0120	0.0144	0.0133
CrohnDisease	mean	0.8486	**0.8520(≈)**	0.7914(+)	0.7759(+)	0.7231(+)	0.7440(+)	0.6993(+)	0.7076(+)
	std	0.0239	0.0101	0.0112	0.0225	0.0194	0.0260	0.0088	0.0107
GLI	mean	**0.9195**	0.9164(≈)	0.8965(+)	0.8510(+)	0.8886(+)	0.8449(+)	0.8176(+)	0.8140(+)
	std	0.0131	0.0146	0.0096	0.0094	0.0198	0.0098	0.0117	0.0119
Leukemia	mean	0.8503	**0.8814(−)**	0.8494(≈)	0.8467(≈)	0.8751(≈)	0.8355(≈)	0.7620(+)	0.7825(+)
	std	0.0151	0.0145	0.0101	0.0181	0.0189	0.0124	0.0122	0.0087
Lung	mean	**0.8197**	0.8184(≈)	0.7974(+)	0.7984(+)	0.8033(≈)	0.7855(+)	0.7645(+)	0.7665(+)
	std	0.0099	0.0093	0.0056	0.0097	0.0081	0.0058	0.0033	0.0055
LungCancer	mean	**0.9444**	0.9272(≈)	0.9344(≈)	0.9128(+)	0.9206(≈)	0.9150(+)	0.8933(+)	0.8911(+)
	std	0.0080	0.0025	0.0033	0.0075	0.0050	0.0079	0.0022	0.0044
MLL	mean	**0.8057**	0.7514(+)	0.7257(+)	0.6057(+)	0.5714(+)	0.6600(+)	0.7386(+)	0.6643(+)
	std	0.0294	0.0249	0.0140	0.0183	0.0156	0.0178	0.0293	0.0205
Orlraws10P	mean	**0.8584**	0.8499(≈)	0.8115(+)	0.7950(+)	0.8174(+)	0.7952(+)	0.7586(+)	0.7626(+)
	std	0.0075	0.0098	0.0112	0.0234	0.0223	0.0126	0.0038	0.0049
Ovarian	mean	0.8432	0.8335(≈)	0.7945(+)	0.7723(+)	**0.8756(−)**	0.7601(+)	0.7384(+)	0.7125(+)
	std	0.0109	0.0080	0.0046	0.0138	0.0137	0.0054	0.0072	0.0036
Prostate	mean	**0.8340**	0.8174(≈)	0.7831(+)	0.7660(+)	0.7914(+)	0.7506(+)	0.6686(+)	0.6759(+)
	std	0.0165	0.0145	0.0136	0.0153	0.0394	0.0223	0.0065	0.0115
SMKCAN	mean	**0.8017**	0.7868(+)	0.7364(+)	0.7700(+)	0.7554(+)	0.7533(+)	0.7080(+)	0.7254(+)
	std	0.0114	0.0082	0.0067	0.0089	0.0111	0.0087	0.0063	0.0053
TOX_171	mean	**0.8539**	0.8245(+)	0.7923(+)	0.8246(+)	0.7603(+)	0.7673(+)	0.6596(+)	0.7563(+)
	std	0.0239	0.0139	0.0147	0.0177	0.0157	0.0178	0.0074	0.0249
Avg. Accuracy		**0.8530**	0.8486	0.8060	0.7989	0.8031	0.7845	0.7428	0.7514

$$MI(X;Y) = \sum_{x\in X}\sum_{y\in Y} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} \tag{1}$$

In Eq.(1), $p(x)$, $p(y)$ and $p(x,y)$ denote the marginal probability distribution of X and Y and their joint probability distribution, respectively. If X and Y are two features, the mutual information MI(X; Y) measures the amount of information shared between the two features. When X is a feature and Y is a class label, the mutual information MI(X; Y) measures the dependence of feature X on class label Y (Lin et al., 2019).

The process of screening features using mutual information is demonstrated in Algorithm 1, which has a hyperparameter k for selecting key features. The choice of values can have a significant impact on the subsequent evaluation. Too high a value may retain redundant features, while too low a value may result in the loss of valuable information, ultimately affecting classification performance. In high-dimensional microarray datasets, 2–5 % of the features are typically retained as the dominant subspace. In this experiment, we set the subspace to 5 % so that irrelevant and redundant features can be filtered out and basic features can be retained for subsequent analysis.

Algorithm 1: Feature filtering based on mutual information

Input: Ω(training dataset), p(number of features), Y (class labels);
Output: W(filtered features)
$W \leftarrow \emptyset$;
for $i = 1$ top **do**
    Calculate MI(X_i, Y) based on Eq(1), where X_i is the ith feature;
end for
$IDX \leftarrow$ Sort(MI);
$W \leftarrow W(IDX\ (1{:}\ p * k))$;
return $\boldsymbol{x}$ (best solution);

2.3. Crow search algorithm

The introduction mentions that CSA has advantages such as simplicity in implementation and a small number of parameters. For instance, PSO requires tuning four parameters: inertia weight, velocity value, social parameter, and cognitive parameter. Similarly, GA requires tuning six parameters: crossover probability, mutation probability, selection operator, crossover method, mutation method, and replacement method. In contrast, CSA only requires tuning two parameters: discovery probability and step size, which significantly reduces the complexity of parameter tuning. Furthermore, CSA has been successfully applied in many engineering fields (Hu et al., 2023; Anter & Ali, 2019).

Based on these advantages, it is reasonable to use the improved CSA as the second stage of the proposed NNS-DFPG-CSA algorithm in this paper, to further evaluate the feature subsets after mutual information filtering. The following section will provide a detailed introduction to the CSA.

CSA relies on the flight patterns and social interactions of crows to capture the dynamics of cooperation and competition between individual crows and groups for optimization.

For the algorithm CSA, each crow can hide its own food and remember the location where it has stored its food. Simultaneously, they attempt to discover the hidden food locations of other crows. Periodically, each crow changes the location where it hides its food. In the context of feature selection models, each crow's position represents a feature subset. Let $\boldsymbol{x}_i^t$ be a vector of length D, where D is the number of features and represents the position of $crow_i$ in the t th iteration. Similarly, $\boldsymbol{m}_i^t$ is also a vector of length D, and represents the memory of $crow_i$. $\boldsymbol{m}_i^t$ primarily stores the best position found by $crow_i$ in the t th iteration. In each iteration, $crow_i$ aims to find the hidden food location of $crow_k$, which is essentially the memory of $crow_k$. Therefore, two scenarios may arise:

Scenario 1: $crow_k$ is unaware of $crow_i$ attempting to steal its food, hence $crow_i$ updates its position as follows:

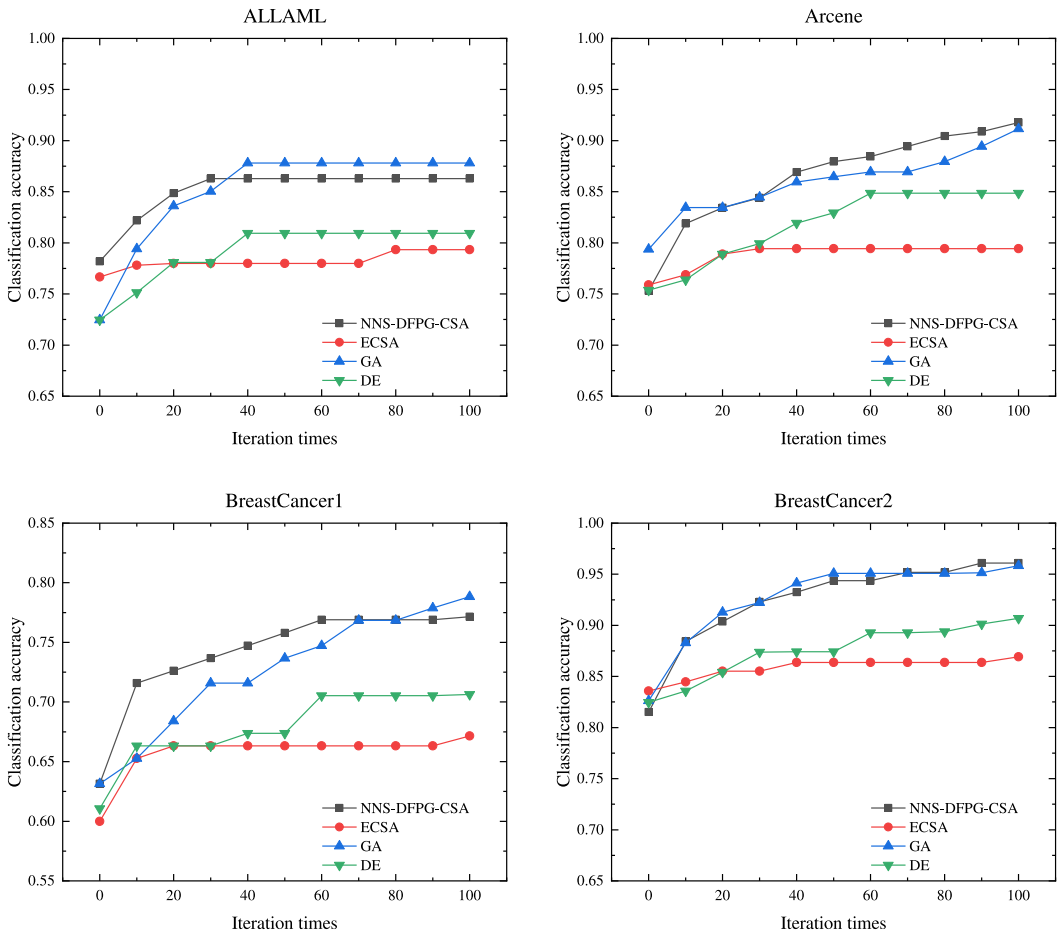


Fig. 4. Comparison of the accuracy of each comparison algorithm.

$$\boldsymbol{x}_i^{t+1} = \boldsymbol{x}_i^t + r_i \times fl_i^t \times \left(\boldsymbol{m}_k^t - \boldsymbol{x}_i^t\right) \tag{2}$$

In *Eq.*(2), fl_i^t represents the flight distance of $crow_i$, r_i is a random number, and $r_i \in [0,1]$. $\boldsymbol{m}_k^t$ is the memory of $crow_k$, and $\boldsymbol{x}_i^t$ is the current position of $crow_i$. *Eq.*(2) primarily signifies the local search of CSA.

Scenario 2: $crow_k$ detects $crow_i$ attempting to locate its hidden food. Therefore, $crow_k$ misleads $crow_i$ to a random location. *Eq.*(3) describes the global search aspect of CSA.

$$\boldsymbol{x}_i^{t+1} = arandomposition \tag{3}$$

Hence, the specific update formula for $crow_i$ is shown as follows:

$$\boldsymbol{x}_i^{t+1} = \begin{cases} \boldsymbol{x}_i^t + r_i \times fl_i^t \times \left(\boldsymbol{m}_k^t - \boldsymbol{x}_i^t\right) & r_i \geq AP_k^t \\ arandomposition & otherwise \end{cases} \tag{4}$$

In *Eq.*(4), r_i is a random number, $r_i \in [0,1]$, and AP_k^t represents $crow_k$'s perception probability in the t th iteration, a parameter that balances global and local search.

In CSA, a fitness function is employed to evaluate the quality of each crow's position (In feature selection, *Fit* is usually denoted as classification accuracy). After each position is updated, the new position is assessed using the fitness function. If the new position after crow's movement is better than the previous one, the crow updates its memory to the new location. Eq. (5) provides a detailed description of this updating process.

$$\boldsymbol{m}_i^{t+1} = \begin{cases} \boldsymbol{x}_i^{t+1} & Fit\left(\boldsymbol{x}_i^{t+1}\right) > Fit\left(\boldsymbol{m}_i^t\right) \\ \boldsymbol{m}_i^t & otherwise \end{cases} \tag{5}$$

When the algorithm reaches the maximum number of iterations, the position of the crow with the best fitness will be retained as the optimal solution. The pseudocode for the Crow Search Algorithm can be found in Algorithm 2.

Algorithm 2: Crow Search Algorithm

Input: N (crow population size), AP (awareness probability), fl (flight length);
Output: $\boldsymbol{x}$ (best solution); //$\boldsymbol{x} = \{x_1, x_2, x_3, \cdots, x_D\}$
Randomly initialize the positions of N crows;
Calculate the fitness of each crow position;
Initialize the memory of each crow;
while termination criterion not fulfilled **do**
  for $i = 1 \rightarrow N$ **do**
    Randomly select a crow position and name it $crow_k$, and $crow_i$ follows $crow_k$;
    Update $crow_i$'s position using *Eq.*(4);
  end for
  Calculate the fitness of each crow position;
  Using *Eq.*(5) to update the memory of crows;
end while
return $\boldsymbol{x}$ (best solution);

The classical Crow Search Algorithm (CSA) faces two main issues:

Imbalance between local and global search.
Excessive randomness in both local and global search, which leads to a low probability of generating excellent solutions.

To address the first issue, we propose time varying dynamic awareness probabilities to balance local and global search. To tackle the second issue, we propose the Elite Neighborhood Search and Dynamic Feature Probability-Guided Crow Search Algorithm (NNS-DFPG-CSA).

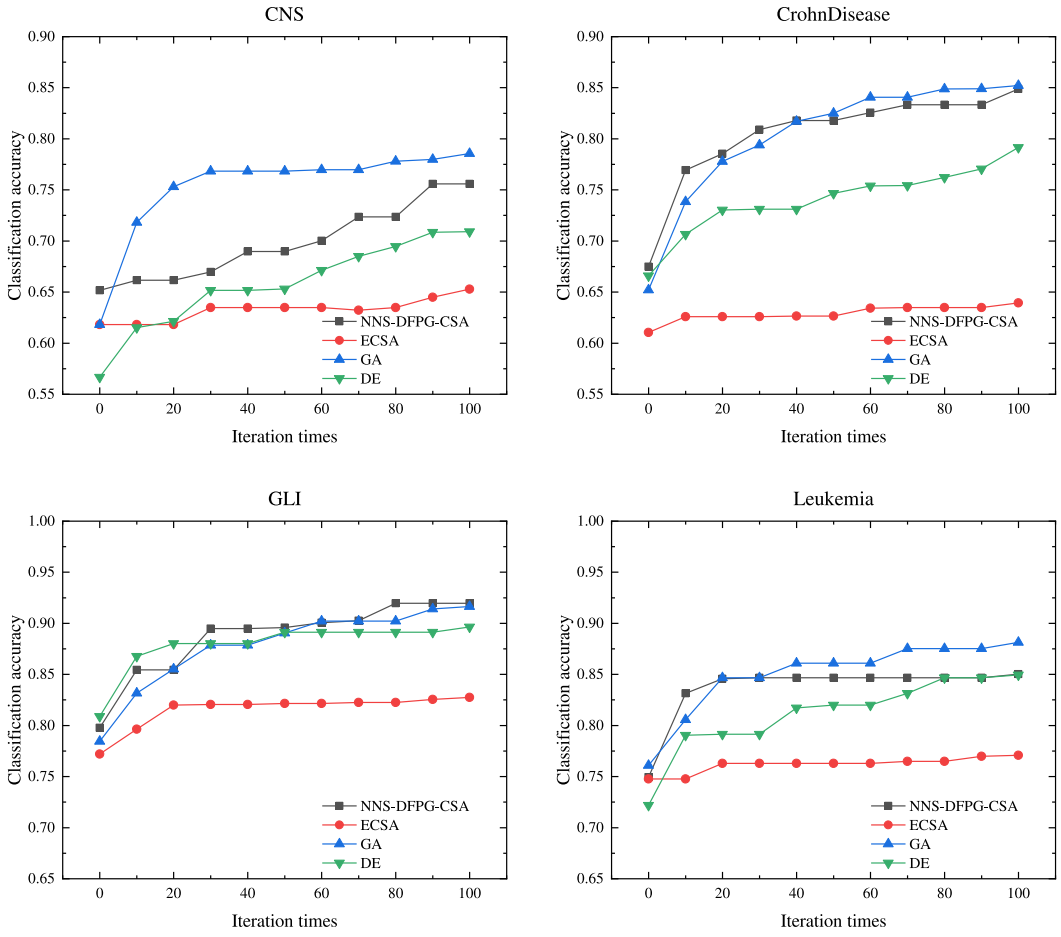


Fig. 5. Comparison of the accuracy of each comparison algorithm.

In the local search strategy of NNS-DFPG-CSA, each crow selects a suitable crow from its own neighbors to follow, and then replaces CSA's random crow selection for following. In the global search strategy, NNS-DFPG-CSA utilizes dynamic feature probabilities to guide crows in generating new positions instead of CSA's random generation of new positions. In this paper, NNS-DFPG-CSA improves both the local and global search capabilities of CSA. Furthermore, feature probabilities are dynamically updated based on the performance of each feature in each iteration. Section 3 will provide a detailed description of the implementation process of NNS-DFPG-CSA.

3. Proposed algorithm

3.1. The framework of NNS-DFPG-CSA

In this section, we will primarily focus on the framework of NNS-DFPG-CSA. Algorithm 3 describes in detail how NNS-DFPG-CSA is updated iteratively.

Algorithm 3: The framework of NNS-DFPG-CSA

Input: N (crow population size);
Output: $\boldsymbol{x}$ (the best solution); //let$\boldsymbol{x} = \{x_1, x_2, x_3, \cdots, x_D\}$
DFP← Initialize the set of dynamic feature probabilities. //*Eq.*(6)
Initialize the positions of N crows based on *DFP*; // Algorithm 4;
while termination criterion not fulfilled **do**
    Fit← Calculate the fitness of each crow position; //*Eq.*(7)
    Obtain the fitness ranking of each crow.
    Obtain *Score* based on *Fit*, and update *DFP* according to *Score*. // Algorithm 5
    Neighbors← Obtain the neighbors of each crow. // Algorithm 6
    for $i = 1 \rightarrow N$ **do**

(*continued on next column*)

(*continued*)

Algorithm 3: The framework of NNS-DFPG-CSA

Input: N (crow population size);
Output: $\boldsymbol{x}$ (the best solution); //let$\boldsymbol{x} = \{x_1, x_2, x_3, \cdots, x_D\}$
DFP← Initialize the set of dynamic feature probabilities. //*Eq.*(6)
Initialize the positions of N crows based on *DFP*; // Algorithm 4;
while termination criterion not fulfilled **do**
    Fit← Calculate the fitness of each crow position; //*Eq.*(7)
    Obtain the fitness ranking of each crow.
    Obtain *Score* based on *Fit*, and update *DFP* according to *Score*. // Algorithm 5
    Neighbors← Obtain the neighbors of each crow. // Algorithm 6
    for $i = 1 \rightarrow N$ **do**
        Calculate the $TVDAP_i^t$ for the i th crow.
        if *rand* < $TVDAP_i^t$
            //Perform local search.
            Case 1: The fitness of $crow_i$ is better than that of all its neighbors.
                Update the position of $crow_i$ using Algorithm 7.

Calculate the $TVDAP_i^t$ for the i th crow.
if *rand* < $TVDAP_i^t$
//Perform local search.
Case 1: The fitness of $crow_i$ is better than that of all its neighbors.
Update the position of $crow_i$ using Algorithm 7.

            Case 2: The fitness of $crow_i$ is not the best compared to its neighbors.
                Update the position of $crow_i$ using Algorithm 8.
        else
            // Perform global search.
            Update the position of $crow_i$ using Algorithm 4.

(*continued on next page*)

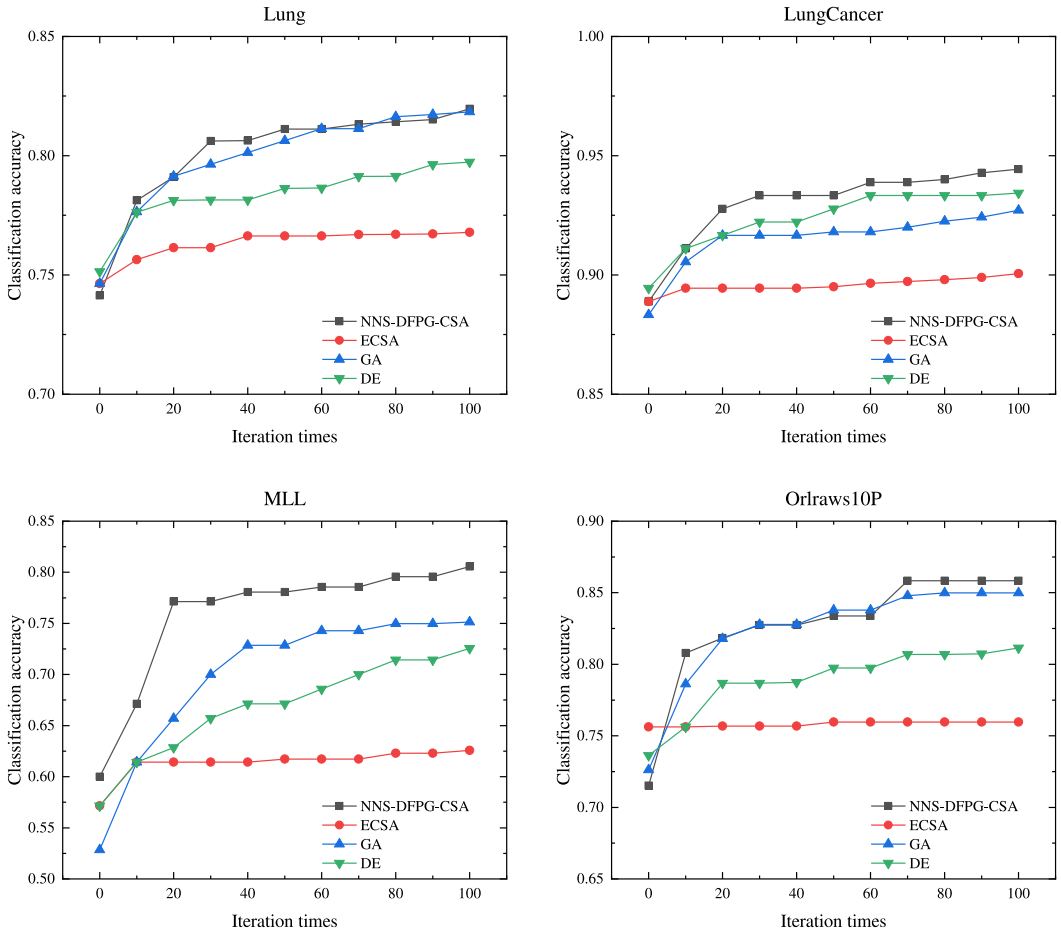


Fig. 6. Comparison of the accuracy of each comparison algorithm.

(*continued*)

Algorithm 3: The framework of NNS-DFPG-CSA

Input: N (crow population size);
Output: $\boldsymbol{x}$ (the best solution); //let$\boldsymbol{x} = \{x_1, x_2, x_3, \cdots, x_D\}$
DFP← Initialize the set of dynamic feature probabilities. //*Eq.*(6)
Initialize the positions of N crows based on *DFP*; // Algorithm 4;
while termination criterion not fulfilled **do**
  Fit← Calculate the fitness of each crow position; //*Eq.*(7)
  Obtain the fitness ranking of each crow.
  Obtain *Score* based on *Fit*, and update *DFP* according to *Score*. // Algorithm 5
  Neighbors← Obtain the neighbors of each crow. // Algorithm 6
  for $i = 1 \rightarrow N$ **do**
    Calculate the $TVDAP_i^t$ for the i th crow.
    if $rand < TVDAP_i^t$
      //Perform local search.
      Case 1: The fitness of $crow_i$ is better than that of all its neighbors.
        Update the position of $crow_i$ using Algorithm 7.
  Merge the new positions of N crows with their old positions and keep the top N positions with the best fitness.
end while
return $\boldsymbol{x}$ (the best solution);

Algorithm 3 outlines the framework of NNS-DFPG-CSA. Firstly, the algorithm initializes the dynamic feature probabilities for D features and place these probabilities into *DFP*, as shown in *Eq.*(6). In *Eq.*(6), γ_j represents the feature probability for the j th feature, where γ_j varies within the range of 0.05 to 0.75. The initial probability for each feature is denoted by φ.

$$DFP = \{\gamma_1, \gamma_2, \cdots, \gamma_D\} \tag{6}$$

In the main loop (while), firstly, the fitness values of N crow positions are calculated, as shown in *Eq.*(2). Secondly, the N crows are ranked based on their fitness. Then, we calculate the feature scores for each feature and obtain *Score*, and then update *DFP* according to *Score*, as detailed in Algorithm 5. Following this, the algorithm obtains m neighbors for each crow by Algorithm 6. Next, we calculate the time-varying dynamic awareness probability ($TVDAP_i^t$) for the i th crow. If $rand < TVDAP_i^t$, the algorithm performs local search; otherwise, does global search. Finally, it shall terminate the algorithm's iterations and return the optimal solution. Fig. 1 illustrates the flow chart of NNS-DFPG-CSA.

3.2. Crow population initialization strategy

In the classic CSA, the position of each crow generated during initialization is represented as a vector consisting of D floating-point numbers, where D is the number of features. Every time the fitness of a crow's position is evaluated, the floating-point vector is converted into a binary vector. The initialization strategy used in this paper is to directly generate binary vectors without the need for binary conversion.

The DFP-based initialization strategy presented in this paper aims to accelerate the identification and in-depth exploration of the potential search space. This strategy prioritizes focusing on those features that have significant potential value for the classification task, thereby effectively reducing unnecessary computational overhead and achieving optimal allocation of computational resources. In particular, this strategy avoids the pitfalls of blind search in the early stages of the iterative process and directs the search paths towards the feature sets that are most likely to improve model performance, thus facilitating the

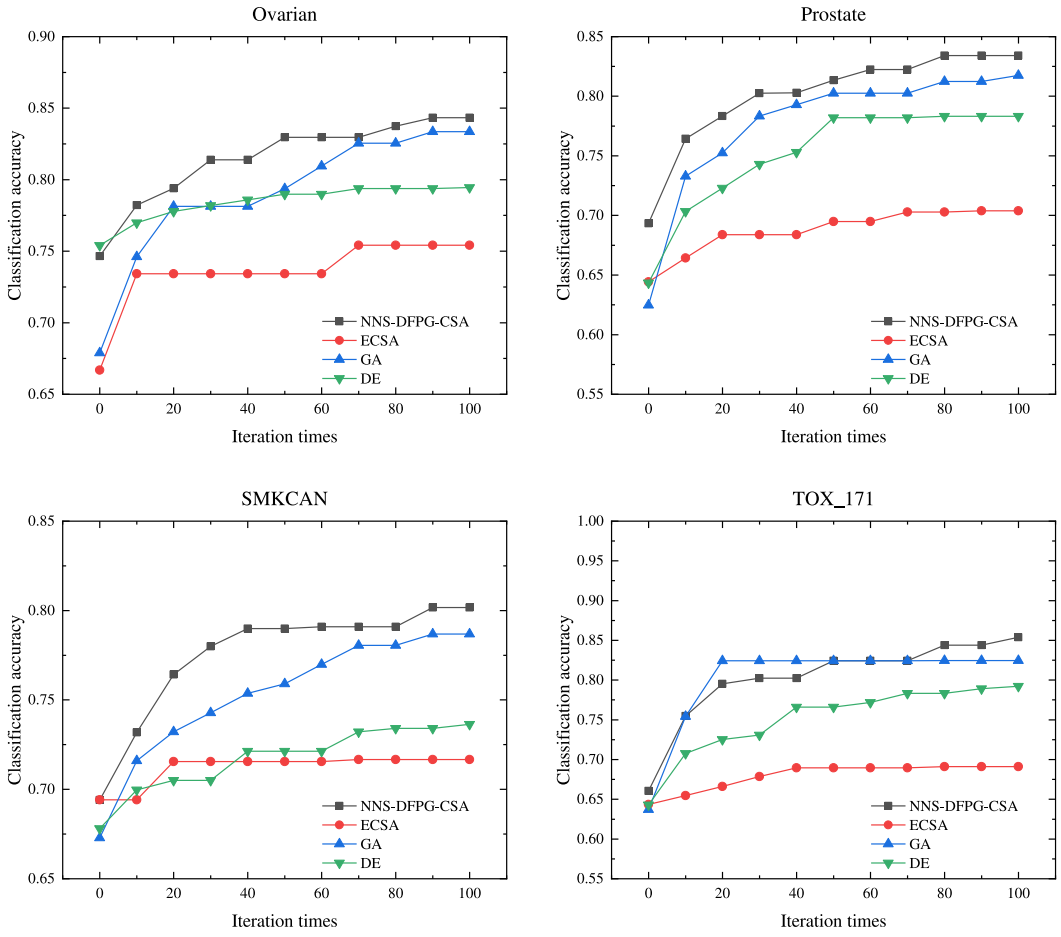


Fig. 7. Comparison of the accuracy of each comparison algorithm.

improvement of classification accuracy and the optimization of model training efficiency.

Algorithm 4 describes the population initialization strategy for NNS-DFPG-CSA. In Algorithm 4, firstly, an $N \times D$ matrix $\boldsymbol{C}$ is generated, where any element of $\boldsymbol{C}$, $x_{ij} = 0or1$. $rand \in [0,1]$ is a random number, and γ_j is the feature probability of the j th feature, $\gamma_j \in [0.05, 0.75]$. When $rand < \gamma_j$, the j th position of the i th crow, $x_{ij} = 1$; otherwise, $x_{ij} = 0$.

Algorithm 4: Initialize the positions of all crows

Input: N (crow population size), D (feature number), DFP(the set of dynamic feature probabilities);

Output: $\boldsymbol{C}$ (The position matrix for N crows.);

$\boldsymbol{C}\leftarrow$ Generate an $N \times D$ matrix.

$$//\boldsymbol{C} = \begin{bmatrix} x_{11} & \cdots & x_{1D} \\ \vdots & \ddots & \vdots \\ x_{N1} & \cdots & x_{ND} \end{bmatrix}, x_{ij} = 0or1, i = 1,2,\cdots,N, j = 1,2,\cdots,D$$

for $i = 1 \rightarrow N$ **do**

  for $j = 1 \rightarrow D$ **do**

    if $rand < \gamma_j$ $//\gamma_j \in DFP$

      $x_{ij} = 1$; $//x_{ij} \in \boldsymbol{C}$

    else

      $x_{ij} = 0$;$//x_{ij} \in \boldsymbol{C}$

  end for

end for

return $\boldsymbol{C}$;

3.3. Update dynamic feature probability

In this section, we will describe the method for updating the feature probabilities for each feature. The numerical values of feature probabilities indicate the likelihood of a feature being selected.

Eq.(7) describes the set *Fit* containing the fitness of N crows, where fit_i represents the fitness of the $crow_i$'s position (i.e., classification accuracy), where $i = 1,2,3,\cdots,N$.

$$Fit = \{fit_1, fit_2, \cdots, fit_N\} \tag{7}$$

Eq.(8) describes the set *Score* containing the scores of D features, where $score_j$ represents the feature score of the j th feature, where $j = 1, 2, 3\cdots, D$.

$$Score = \{score_1, score_2, \cdots, score_D\} \tag{8}$$

Eq.(9) describes how to calculate the feature score. $score_j$ represents the average fitness among all crows in which the j th feature appears. In *Eq.*(9), fit_i represents the fitness value of the i th crow. x_{ij} represents the j th element of the i th crow, and $x_{ij} = 0or1$.

$$score_j = \frac{\sum_{i=1}^{N} fit_i \times x_{ij}}{\sum_{i=1}^{N} x_{ij}} \quad fit_i \in Fit, score_j \in Score, x_{ij} \in C \tag{9}$$

Fig. 2 provides a detailed description of the feature score calculation process. For the first feature, $F1$, which appears 2 times among 4 crows, the feature score of $F1$ is calculated as $score_1 = (0.81 + 0.93)/2$.

Algorithm 5 describes how to update the dynamic feature probabilities. First, calculate the feature scores for all features. Then, sort the features based on their feature scores. Finally, update the dynamic feature probabilities based on the feature scores of each feature. In Algorithm 5, ξ is an adjustment value, and $\xi = 5/T$. T represents the number of iterations in the algorithm.

In the process of dynamically adjusting the probability of feature selection, it aims to identify and screen out those features that are critical for significantly improving the performance of the model. This mechanism dynamically adjusts the probability of features being

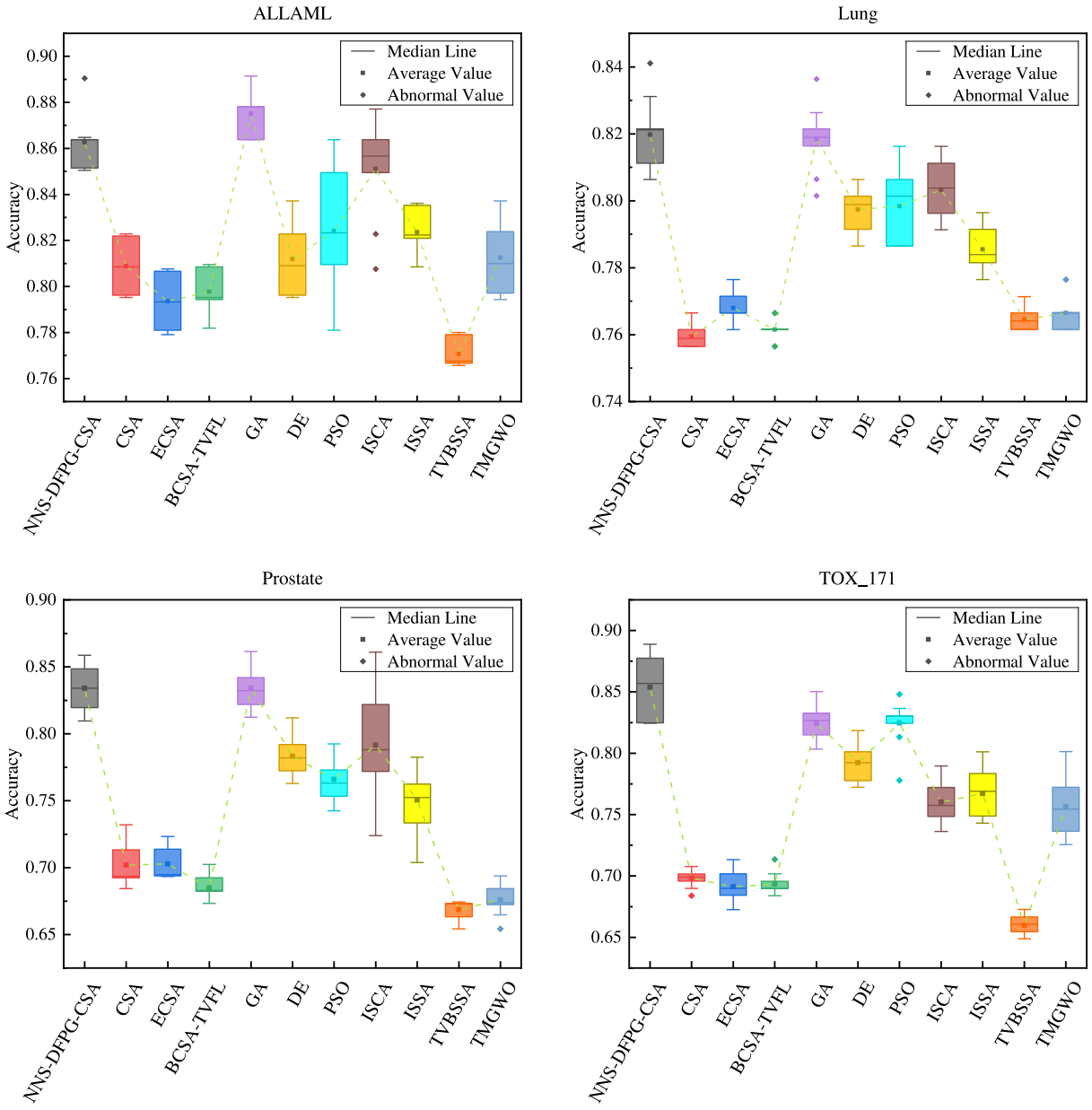


Fig. 8. Analyzing the stability of each algorithm through box plots.

selected by continuously monitoring and evaluating their performance in model training, with a view to increasing the chances of crows selecting these efficient features in subsequent iterations.

Algorithm 5: Update dynamic feature probability
Input: *Fit*(the fitness of each crow position), *DFP*(the set of dynamic feature probabilities);
Output: *DFP*;
Score← Calculate feature scores for all features. // *Eq.*(7)-(9)
Sort the features based on *Score*.
Increase the dynamic feature probability for the top 50 % of features with higher score and decrease it for the bottom 50 % of features with lower score, with an adjustment value of ξ.
return *DFP*(updated);

3.4. Elite neighborhood search strategy

In the classical Crow Search Algorithm, the local search strategy involves $crow_i$ following a randomly selected crow to obtain a new solution. However, this approach has a drawback that it randomly selects a crow to follow. Thus if the chosen crow has lower fitness, a local search in its vicinity may reduce the likelihood of obtaining excellent solutions.

Therefore, in the local search strategy of NNS-DFPG-CSA, we have dispensed with the random selection of crows to follow. Instead, we choose the crow neighbor with the best fitness for each crow. In this section, we will focus on the neighbor search strategy of NNS-DFPG-CSA, specifically how to find the neighbors for each crow.

Algorithm 6: Obtain the neighbors of each crow
Input: ***C*** (The position matrix for N crows.);
Output: *Neighbors*;
Neighbors← Create an empty set.
for $i = 1 \rightarrow N$ **do**
    neighbor← Create an empty set.
    Calculate the similarity between $crow_i$ and the remaining $N-1$ crows. //*Eq.*(10)
    Find the 4 crows most similar to $crow_i$ and place them into the *neighbor*.
    Place *neighbor* into set *Neighbors*.
return *Neighbors*;

Table 5
Comparison of NNS-DFPG-CSA and CSA based methods in terms of average and std of FSR.

Dataset	Stat. measure	NNS-DFPG-CSA	CSA based		
			CSA	ECSA	BCSA-TVFL
ALLAML	mean	**0.0079**	0.0235 (+)	0.0262 (+)	0.0366 (+)
	std	0.0128	0.0043	0.0125	0.0189
Arcene	mean	**0.0076**	0.0249 (+)	0.0306 (+)	0.0363 (+)
	std	0.0214	0.0182	0.0758	0.0161
BreastCancer1	mean	**0.0054**	0.0248 (+)	0.0149 (+)	0.0362 (+)
	std	0.0297	0.0101	0.0969	0.0087
BreastCancer2	mean	**0.0067**	0.0246 (+)	0.0304 (+)	0.0364 (+)
	std	0.0147	0.0100	0.0127	0.0096
CNS	mean	**0.0055**	0.0246 (+)	0.0205 (+)	0.0359 (+)
	std	0.0294	0.0109	0.0526	0.0146
CrohnDisease	mean	**0.0062**	0.0250 (+)	0.0331 (+)	0.0357 (+)
	std	0.0296	0.0109	0.0433	0.0096
GLI	mean	**0.0059**	0.0251 (+)	0.0260 (+)	0.0362 (+)
	std	0.0060	0.0127	0.0790	0.0108
Leukemia	mean	**0.0066**	0.0244 (+)	0.0267 (+)	0.0357 (+)
	std	0.0075	0.0142	0.0109	0.0325
Lung	mean	**0.0078**	0.0246 (+)	0.0260 (+)	0.0359 (+)
	std	0.0063	0.0075	0.0615	0.0171
LungCancer	mean	**0.0072**	0.0245 (+)	0.0304 (+)	0.0356 (+)
	std	0.0088	0.0136	0.0596	0.0128
MLL	mean	**0.0057**	0.0253 (+)	0.0215 (+)	0.0359 (+)
	std	0.0116	0.0163	0.0378	0.0055
Orlraws10P	mean	**0.0084**	0.0253 (+)	0.0285 (+)	0.0364 (+)
	std	0.0046	0.0087	0.0265	0.0154
Ovarian	mean	**0.0051**	0.0251 (+)	0.0082 (+)	0.0366 (+)
	std	0.0283	0.0136	0.1253	0.0071
Prostate	mean	**0.0055**	0.0250 (+)	0.0283 (+)	0.0363 (+)
	std	0.0128	0.0128	0.1256	0.0106
SMKCAN	mean	**0.0061**	0.0249 (+)	0.0277 (+)	0.0363 (+)
	std	0.0271	0.0146	0.0634	0.0107
TOX_171	mean	**0.0091**	0.0218 (+)	0.0277 (+)	0.0311 (+)
	std	0.0146	0.0126	0.0119	0.0219
Avg. FSR		**0.0067**	0.0246	0.02542	0.0358

Algorithm 6 provides a detailed description of the process that each crow obtains its own neighbors. The first step is to create an extensive set called *"Neighbors"*. Within an iterative loop, the process is shown as follows: firstly, a special set is instantiated to hold the neighbors of $crow_i$, and then to compute the similarity between $crow_i$ and the remaining N-1 crows. The four crows with the highest similarity scores are then added to the *"neighbor"* set. Finally, the *"neighbor"* sets for all N crows are added to the *"Neighbors"* set, and the loop concludes.

Eq.(10) describes how to calculate the similarity between two crows. $crow_m$ and $crow_n$ are vectors of length D. $crow_m = \{x_1, x_2, x_3, \cdots, x_D\}$, where $x_i = 0 or 1$, $i = 1,2,3,\cdots,D$. $crow_n = \{y_1, y_2, y_3, \cdots, y_D\}$, where $y_i = 0 or 1$, $i = 1,2,3,\cdots,D$.

$$S(crow_m, crow_n) = \frac{D - \sum_{i=1}^{D} |x_i - y_i|}{D} \tag{10}$$

In *Eq.*(10), D represents the number of features. A larger value of $S(crow_m, crow_n)$ means that $crow_m$ and $crow_m$ are more similar, where $S(crow_m, crow_n) \in [0,1]$.

3.5. Time varying dynamic awareness probability (TVDAP)

The main idea of the TVDAP method proposed in this paper is that crows with physically strong fitness are more inclined to perform global search. Crows tend to contribute their existing food sources to the population and search for better ones. Crows with lower fitness levels are more likely to perform local search. In the classic CSA, the *AP* parameter is typically set to a constant value 0.1. When the TVDAP method is applied to CSA, the *AP* parameter of each crow changes based on their ranking in the population and the current iteration count of the algorithm. The specific steps of this method are shown as follows:

(1) Calculate the fitness values of all crows in the population.
(2) Sort the fitness of all crows.
(3) The AP value of $crow_i$ is calculated using *Eq.*(11).

$$TVDAP_i^t = AP_{min}^t + \left(AP_{max}^t - AP_{min}^t\right) * \frac{\lambda_i}{N} \# \tag{11}$$

In *Eq.*(11), λ_i represents the ranking of $crow_i$ among all crows, and AP_{min}^t and AP_{max}^t represent the minimum and maximum *AP* values at the t th iteration, respectively. The initial values for AP_{min} and AP_{max} are set to 0.1 and 0.8, respectively. The update scheme for AP_{min} and AP_{max} is as follows:

$$AP_{min}^{t+1} = AP_{min}^t + 0.3 * \frac{1}{T} \tag{12}$$

$$AP_{max}^{t+1} = AP_{max}^t - 0.3 * \frac{1}{T} \tag{13}$$

In *Eq.*(12) and *Eq.*(13), T represents the number of iterations of the algorithm. As seen from *Eq.*(11), crows with higher rankings have smaller *TVDAP* values, and vice versa. Therefore, by comparing the *rand* and *TVDAP* values, crows can be divided into two categories, where $rand \in [0,1]$. When $rand < TVDAP_i$, $crow_i$ performs local search, otherwise, does global search. With the increase in the number of iterations, AP_{min} continually increases, while AP_{max} decreases. The range of *TVDAP* values will also be narrowed. Consequently, crows with strong fitness have a higher probability of performing global search that leads to the discovery of better solutions.

3.6. Proposed local search approach

In CSA, each crow randomly selects another crow to follow, which means they gather some feature selection information from other crows. This constitutes CSA's local search strategy. The local search strategy proposed in this paper can be divided into two cases:

Case (1): When $crow_i$'s fitness value is better than that of all its neighbors, Algorithm 7 is used to update $crow_i$'s position.

Case (2): When $crow_i$'s fitness value is not the best one compared to all its neighbors, Algorithm 8 is used to update $crow_i$'s position.

Algorithm 7: Update the position of $crow_i$. // **Case (1)**

Input: $crow_i$, *DFP*(the set of dynamic feature probabilities), D(the number of features);

Output: $crow_{new}$;
Generate a random number k, where $k \in [1, D/100]$. // if $D < 100$, $k = 1$.
$crow_{new} \leftarrow crow_i$;
$crow_\alpha \leftarrow$ Create a vector of length D filled with zeros.
$crow_\beta \leftarrow$ Create a vector of length D filled with ones.
$O \leftarrow crow_{new} \bigcap crow_\alpha$;
$P \leftarrow crow_{new} \bigcap crow_\beta$;
select the k features with the highest feature probabilities from O and change the values at the corresponding positions in $crow_{new}$ from 0 to 1;
select the k features with the lowest feature probabilities from P and change the values at the corresponding positions in $crow_{new}$ from 1 to 0;
return $crow_{new}$;

Table 6
Comparison of NNS-DFPG-CSA and other methods in terms of average and std of FSR.

Datasets	Stat.measure	NNS-DFPG-CSA	Other Method						
			GA	DE	PSO	ISCA	ISSA	TVBSSA	TMGWO
ALLAML	mean	0.0079	0.0241(+)	0.0250(+)	0.0237(+)	**0.0048(≈)**	0.0253(+)	0.0197(+)	0.0198(+)
	std	0.0128	0.0119	0.0106	0.0083	0.0189	0.0301	0.0208	0.0112
Arcene	mean	**0.0076**	0.0251(+)	0.0252(+)	0.0241(+)	0.0084(≈)	0.0245(+)	0.0211(+)	0.0246(+)
	std	0.0214	0.0142	0.0204	0.0140	0.0251	0.0143	0.0190	0.0664
BreastCancer1	mean	0.0054	0.0249(+)	0.0251(+)	0.0249(+)	**0.0053(≈)**	0.0246(+)	0.0206(+)	0.0247(+)
	std	0.0297	0.0117	0.0112	0.0133	0.0537	0.0112	0.0162	0.0706
BreastCancer2	mean	**0.0067**	0.0252(+)	0.0247(+)	0.0248(+)	0.0078(≈)	0.0249(+)	0.0202(+)	0.0248(+)
	std	0.0147	0.0072	0.0134	0.0098	0.0198	0.0085	0.1294	0.0701
CNS	mean	0.0055	0.0237(+)	0.0232(+)	0.0245(+)	**0.0051(≈)**	0.0248(+)	0.0169(+)	0.0245(+)
	std	0.0294	0.0107	0.0191	0.0173	0.0264	0.0119	0.0207	0.0775
CrohnDisease	mean	**0.0062**	0.0246(+)	0.0245(+)	0.0247(+)	0.0075(≈)	0.0253(+)	0.0212(+)	0.0249(+)
	std	0.0296	0.0090	0.0131	0.0128	0.0259	0.0091	0.0230	0.1452
GLI	mean	0.0059	0.0252(+)	0.0247(+)	0.0243(+)	**0.0043(≈)**	0.0244(+)	0.0198(+)	0.0244(≈)
	std	0.0060	0.0152	0.0127	0.0111	0.0359	0.0148	0.0212	0.0580
Leukemia	mean	0.0066	0.0244(+)	0.0244(+)	0.0245(+)	**0.0043(≈)**	0.0259(+)	0.0209(+)	0.0241(+)
	std	0.0075	0.0106	0.0112	0.0125	0.0179	0.0234	0.0154	0.0172
Lung	mean	0.0078	0.0237(+)	0.0239(+)	0.0244(+)	**0.0054(≈)**	0.0251(+)	0.0202(+)	0.252(+)
	std	0.0063	0.0162	0.0104	0.0124	0.0124	0.0058	0.0244	0.1062
LungCancer	mean	**0.0072**	0.0242(+)	0.0243(+)	0.0246(+)	0.0083(≈)	0.0254(+)	0.0212(+)	0.0249(+)
	std	0.0088	0.0150	0.0115	0.0163	0.0555	0.0134	0.0345	0.0837
MLL	mean	0.0057	0.0245(+)	0.0250(+)	0.0243(+)	**0.0040(≈)**	0.0246(+)	0.0205(+)	0.0251(+)
	std	0.0116	0.0149	0.0296	0.0114	0.0289	0.0172	0.0114	0.0169
Orlraws10P	mean	0.0084	0.0241(+)	0.0247(+)	0.0246(+)	**0.0054(≈)**	0.0243(+)	0.0197(+)	0.0254(+)
	std	0.0046	0.0179	0.0176	0.0124	0.0234	0.0128	0.0102	0.0233
Ovarian	mean	0.0051	0.0244(+)	0.0247(+)	0.241(+)	**0.0030(≈)**	0.0244(+)	0.0181(+)	0.0249(+)
	std	0.0283	0.0112	0.0162	0.0109	0.0721	0.0100	0.1026	0.1787
Prostate	mean	0.0055	0.0238(+)	0.0243(+)	0.0238(+)	**0.0038(≈)**	0.0247(+)	0.0204(+)	0.0244(+)
	std	0.0128	0.0155	0.0217	0.0105	0.0575	0.0109	0.0273	0.1656
SMKCAN	mean	**0.0061**	0.0244(+)	0.0248(+)	0.0250(+)	0.0069(≈)	0.0252(+)	0.0185(+)	0.0250(+)
	std	0.0271	0.0160	0.0137	0.0139	0.0188	0.0082	0.0091	0.1031
TOX_171	mean	0.0091	0.0196(+)	0.0201(+)	0.0206(+)	**0.0066(≈)**	0.0213(+)	0.0185(+)	0.0186(+)
	std	0.0146	0.0154	0.0034	0.0146	0.0156	0.0129	0.0254	0.0344
Avg. FSR		0.0067	0.0241	0.0243	0.0377	**0.0057**	0.0247	0.0198	0.0383

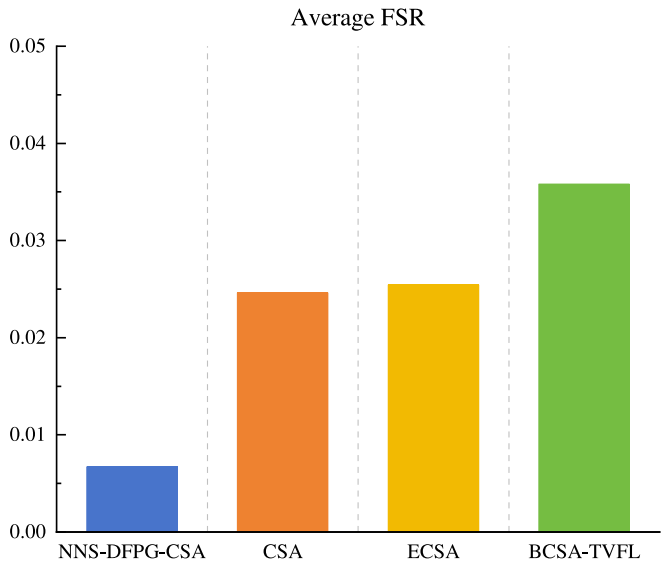


Fig. 9. Average FSR statistic results of the proposed algorithm and the CSA-based algorithm.

Algorithm 7 describes the way to update $crow_i$'s position when $crow_i$'s fitness is better than all its neighbors. In the first step, a random number k is generated, where k ranges from 1 to $D/100$. When D is less than 100, k is set to 1 directly, where D represents the number of features. In the second step, $crow_i$'s sequence is duplicated to create a new crow called $crow_{new}$. In the third step, from the $crow_{new}$'s sequence, the features with values equal to 1 are identified, and the k features with the lowest feature probabilities are selected to change their values to 0. In the fourth step, from the $crow_{new}$'s sequence, the features with values equal to 0 are identified, and the k features with the highest feature probabilities are selected to change their values to 1. The main purpose of the strategy shown in case (1) is that if the crows are better adapted than the neighboring crows, the feature selection probability is used as a benchmark and the crows are adapted to hopefully explore the area more deeply.

Algorithm 8: Update the position of $crow_i$. // **Case (2)**

Input: P, $crow_i$, DFP(the set of dynamic feature probabilities), D(the number of features);
Output: $crow_{new}$;
$crow_m, crow_n \leftarrow$ Select two crows with the best fitness from the neighbors of $crow_i$.
$crow_{new} \leftarrow$ Create a vector of length D filled with zeros.
$U \leftarrow crow_i \bigcap crow_m \bigcap crow_n$// Select the feature indexes, where all of the three crows have either all 1 s or all 0 s on these indexes.
$V \leftarrow crow_m \bigcap crow_n - U$
$W \leftarrow crow_m \bigcup crow_n - crow_m \bigcap crow_n$
$crow_{new}[U] \leftarrow crow_i[U]$
for element $i in V$ **do**
  if $rand < 0.66$ // $rand$ is a random number,$rand \in [0,1]$
    $crow_{new}[i] \leftarrow crow_m[i]$
  else
    if $crow_m[i] = 0$
      $crow_{new}[i] \leftarrow 1$
    else
      $crow_{new}[i] \leftarrow 0$
for element $j in W$ **do**
  if $rand < 0.5$
    $crow_{new}[i] \leftarrow 1$
  else
    $crow_{new}[i] \leftarrow 0$
return $crow_{new}$;

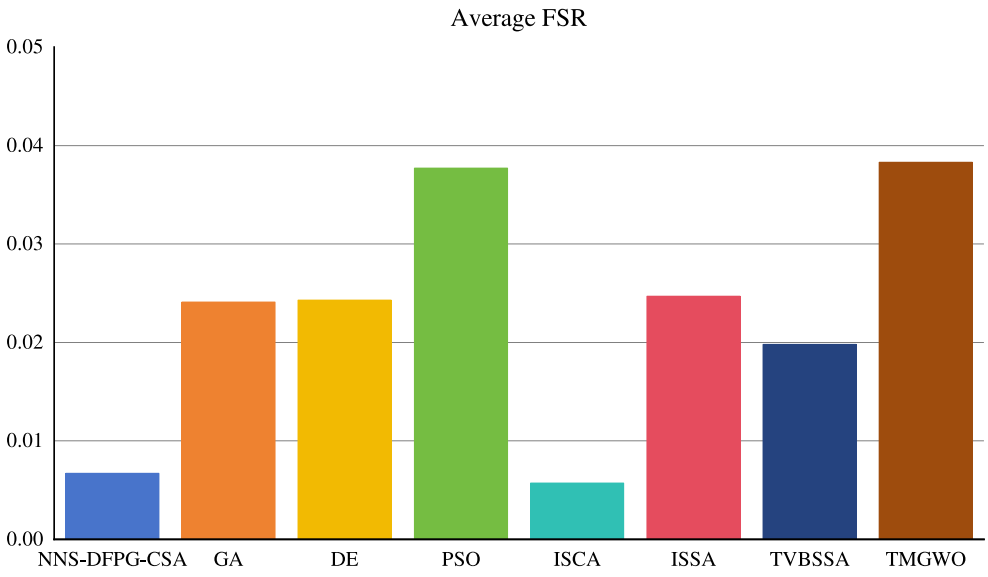


Fig. 10. Average FSR statistics of the proposed algorithm and other compared algorithms.

Algorithm 8 outlines the method for updating $crow_i$ when the fitness value of $crow_i$ is lower than that of its neighbors. Firstly, the two crows with the highest fitness among $crow_i$'s neighbors, $crow_m$ and $crow_n$, are selected. Next, $crow_{new}$ is generated, where each element in $crow_{new}$ is set to 0. U represents the indices in the sequences of $crow_m$, $crow_n$, and $crow_i$ where the values at the same positions are either all 1 or all 0. V represents the indices in the sequences of $crow_m$ and $crow_n$ where the values at the same positions are either all 1 or all 0, and then the indices in U are excluded. W represents the indices in the sequences of $crow_m$ and $crow_n$ where the values at the same positions are different (i.e., one is 0, and the other is 1). Then, by using U, the corresponding indices elements from $crow_i$ are assigned to $crow_{new}$. For feature indices in V, there is a 0.66 probability of assigning the corresponding indices elements from $crow_m$ to $crow_{new}$, and a 0.33 probability of random assignment as 0 or 1. For indices in W, the random values of 0 or 1 are assigned to the corresponding positions in $crow_{new}$. The final step is to return $crow_{new}$.

The strategy adopted in case (2) is simplified and illustrated in Fig. 3. The core idea of the strategy proposed in this paper is that, for the features located in the common feature region U, we directly and preferentially assign these features to the new crow model, given their significant contribution to improving the model performance. Immediately afterwards, for the features in region V, we decided to include them in the construction of the new crow model with a higher probability, since they also have a non-negligible potential. As for the features in region W, in order to fully and adequately explore the possibilities of this region, we adopted a neutral attitude, i.e. we assigned the same probability of selection to these features in order to decide whether or not to include them in the final model construction process.

4. Experimentation and discussion

In this Section, in order to verify the performance of NNS-DPG-CSA, we compare it with ten other feature selection algorithms. These algorithms include the original CSA, the CSA improved by other researchers, and some of the current mainstream metaheuristics. In addition, classical algorithms such as GA, DE, and PSO were selected as controls. These classical algorithms are chosen because many improved versions of GA, PSO, and DE are usually optimized for specific problems or scenarios, and it is difficult to fully reflect the overall performance of such algorithms. In contrast, the classical versions of GA, PSO, and DE have been extensively studied, and their performance curves and optimization characteristics are clearer and can be used as a standard control group. This choice helps to demonstrate more intuitively the advantages of the NNS-DPG-CSA proposed in this paper in solving a wide range of high-dimensional feature selection problems.

We will cover experimental details, including datasets and algorithm parameter configurations. Following that, we'll present various evaluation metrics such as average accuracy, fitness, and feature selection rate. Finally, a comparison will be conducted between NNS-DFPG-CSA and comparison algorithms using additional metrics like runtime, standard deviation, and the Wilcoxon rank-sum test (Derrac et al., 2011) (with a significance level set at 0.05).

4.1. Datasets

We opted for 16 gene microarray datasets (J. Li et al., 2017; Burczynski et al., 2006; Zhu et al., 2007; Chowdary et al., 2006) to assess the performance of NNS-DFPG-CSA. The details of these datasets, such as the count of features, samples, and class labels, are presented in Table 1, labeled as "No. of total features", "No. of samples", and "No. of classes", respectively.

4.2. Parameter settings

The values of the parameters for all comparison algorithms can be found in Table 2. The parameters of the comparison algorithms are all based on the specific values recommended in the original paper. The parameter information for NNS-DFPG-CSA is given in the last row of Table 2. Regarding the common parameters, all datasets use Mutual information for feature selection, selecting the top 5 % of features that are highly correlated with the class labels. All algorithms use the same classifier, namely the KNN classifier, to evaluate algorithm performance. The number of algorithm iterations is 100, and the number of individuals in the population is 100. All experimental results are the average of ten runs.

4.3. Evaluation metrics

In this paper, we have used three renowned metrics to assess the proposed feature selection algorithm. These three metrics are shown as follows:

Table 7
Comparison of NNS-DFPG-CSA and CSA based methods in terms of average and std of fitness.

Dataset	Stat. measure	NNS-DFPG-CSA	CSA based		
			CSA	ECSA	BCSA-TVFL
ALLAML	mean	**0.1244**	0.1744 (+)	0.1884 (+)	0.1857 (+)
	std	0.0038	0.0048	0.0098	0.0065
Arcene	mean	**0.0747**	0.1899 (+)	0.1860 (+)	0.1907 (+)
	std	0.0091	0.0038	0.0128	0.0028
BreastCancer1	mean	**0.2061**	0.3075 (+)	0.2971 (+)	0.3456 (+)
	std	0.0055	0.0082	0.0181	0.0048
BreastCancer2	mean	**0.0358**	0.1691 (+)	0.1208 (+)	0.1592 (+)
	std	0.0027	0.0010	0.0013	0.0010
CNS	mean	**0.2203**	0.3236 (+)	0.3144 (+)	0.3363 (+)
	std	0.0133	0.0078	0.0210	0.0074
CrohnDisease	mean	**0.1369**	0.3062 (+)	0.3279 (+)	0.3282 (+)
	std	0.0062	0.0047	0.0082	0.0060
GLI	mean	**0.0730**	0.1342 (+)	0.1579 (+)	0.1872 (+)
	std	0.0050	0.0033	0.0128	0.0040
Leukemia	mean	**0.1354**	0.2181 (+)	0.2090 (+)	0.1951 (+)
	std	0.0021	0.0055	0.0152	0.0033
Lung	mean	**0.1631**	0.2189 (+)	0.2115 (+)	0.2182 (+)
	std	0.0019	0.0026	0.0086	0.0026
LungCancer	mean	**0.0508**	0.0909 (+)	0.0925 (+)	0.1006 (+)
	std	0.0009	0.0022	0.0078	0.0019
MLL	mean	**0.1754**	0.3998 (+)	0.3390 (+)	0.4189 (+)
	std	0.0058	0.0058	0.0088	0.0051
Orlraws10P	mean	**0.1283**	0.2152 (+)	0.2191 (+)	0.2271 (+)
	std	0.0024	0.0025	0.0073	0.0025
Ovarian	mean	**0.1416**	0.2238 (+)	0.2221 (+)	0.2239 (+)
	std	0.0028	0.0014	0.0156	0.0016
Prostate	mean	**0.1500**	0.2707 (+)	0.2703 (+)	0.2873 (+)
	std	0.0050	0.0037	0.0242	0.0041
SMKCAN	mean	**0.1791**	0.2776 (+)	0.2577 (+)	0.2547 (+)
	std	0.0059	0.0062	0.0162	0.0036
TOX_171	mean	**0.1324**	0.2738 (+)	0.2807 (+)	0.2791 (+)
	std	0.0042	0.0071	0.0214	0.0011
Avg. fitness		**0.1330**	0.2371	0.2309	0.2461

4.3.1. Average accuracy

This metric describes the average classification accuracy after an algorithm runs k times and is shown as follows:

$$Averageaccuracy = \frac{1}{k}\sum_{i=1}^{k} Acc_i \tag{14}$$

where Acc_i represents the accuracy for the i th convergence.

4.3.2. Average feature selection ratio

The average feature selection rate (FSR) is the average ratio of selected features to all features after an algorithm has been run k times, and is defined as follows:

$$FSR = \frac{1}{k}\sum_{i=1}^{k} \frac{L_s^i}{L_{all}} \tag{15}$$

In *Eq.*(15), L_s^i signifies the count of selected features within the optimal solution of the i th run, whereas L_{all} denotes the total number of features.

4.3.3. Average fitness

This metric describes the average fitness after an algorithm runs k times:

$$Averagefitness = \frac{1}{k}\sum_{i=1}^{k} fit_i \tag{16}$$

where fit_i represents the best fitness for the i th convergence.

The fitness is shown as follows:

$$fitness = (1-\zeta) \times err + \zeta \times \frac{L_s}{L_{all}} \tag{17}$$

The utilization of a fitness function is motivated by the limitations of relying solely on accuracy metrics for a holistic assessment of the algorithm's performance. The quantity of selected features plays a pivotal role in algorithm evaluation. For *Eq.*(17), err denotes the classification error rate, and ζ is a proportionality factor connecting the classification error rate to the number of selected features, where $\zeta = 0.1$. A decreased fitness value in *Eq.*(17) means a reduction in the algorithm's classification error rate and a more limited selection of features.

4.4. NNS-DFPG-CSA overall performance

4.4.1. Accuracy metric

The primary goal of feature selection is to improve the classification accuracy, and in this paper, we use the accuracy metric to evaluate the quality of candidate feature subsets. The accuracy rates of all algorithms and their standard deviations are listed in Tables 3 and 4, where the symbols "+", "≈" and "-" indicate that the results of NNS- DFPG-CSA results are better than, close to, or inferior to the comparison algorithms, respectively.

According to the results in Table 3, NNS-DFPG-CSA has higher classification accuracies than the classical CSA and its improved versions on all 16 high-dimensional feature selection datasets. Table 4 shows that the algorithm proposed in this paper achieves the highest average classification accuracy on 10 datasets compared to other metaheuristics.

The line plots of classification accuracy with the number of iterations in Figs. 4–7 are further analyzed to compare the performance of the two algorithms (GA and DE) with higher average accuracies, as well as ECSA, on the 16 datasets. Notably, NNS-DFPG-CSA has higher initial classification accuracies on most of the datasets compared to ECSA, indicating the significant effectiveness of the dynamic feature probability-guided crow location initialization strategy. The convergence curves on the BreastCancer2 (Fig. 4), GLI (Fig. 5), LungCancer (Fig. 6), and SMKCAN (Fig. 7) datasets further confirm that the elite neighborhood search strategy for improving local search, along with the dynamic feature probability-guided global search strategy, can effectively prevent the algorithm from being trapped in local optima.

Fig. 8 presents the accuracy box plots of all comparison algorithms on four benchmark datasets. Overall, the proposed algorithm demonstrates a more concentrated data distribution, along with higher mean and median accuracy. Notably, although NNS-DFPG-CSA produces high-accuracy outliers on the ALLALM and Lung datasets, this indirectly highlights its potential to escape local optima.

In summary, the proposed NNS-DFPG-CSA algorithm exhibits significant advantages in classification accuracy when handling high-dimensional feature selection tasks, providing an efficient and reliable solution for feature selection in complex data scenarios.

4.4.2. Feature selection ratio (FSR) metric

The second goal of feature selection is to reduce the number of features and hence the computational cost. With the increasing size and complexity of data, feature selection has become a critical aspect of the data mining task.

Table 8
Comparison of NNS-DFPG-CSA and other methods in terms of average and std of fitness.

Datasets	Stat.measure	NNS-DFPG-CSA	Other Method						
			GA	DE	PSO	ISCA	ISSA	TVBSSA	TMGWO
ALLAML	mean	0.1244	**0.1148(≈)**	0.1718(+)	0.1608(≈)	0.1345(≈)	0.1614(≈)	0.2083(+)	0.1707(+)
	std	0.0038	0.0092	0.0072	0.0035	0.0064	0.0032	0.0025	0.0089
Arcene	mean	**0.0747**	0.0821(≈)	0.1387(+)	0.1299(+)	0.1618(+)	0.1557(+)	0.1918(+)	0.1688(+)
	std	0.0091	0.0027	0.0070	0.0040	0.0105	0.0047	0.0025	0.0115
BreastCancer1	mean	0.2061	**0.1929(≈)**	0.2668(+)	0.2555(+)	0.2335(≈)	0.2952(+)	0.3336(+)	0.3114(+)
	std	0.0055	0.0069	0.0051	0.0120	0.0240	0.0111	0.0050	0.0243
BreastCancer2	mean	**0.0358**	0.0400(≈)	0.0862(+)	0.0727(+)	0.0862(+)	0.0966(+)	0.1534(+)	0.1528(+)
	std	0.0027	0.0007	0.0028	0.0042	0.0045	0.0009	0.0146	0.0126
CNS	mean	0.2203	**0.1954(−)**	0.2641(+)	0.2356(≈)	0.2269(≈)	0.2572(+)	0.3143(+)	0.3204(+)
	std	0.0133	0.0149	0.0019	0.0088	0.0176	0.0071	0.0105	0.0246
CrohnDisease	mean	0.1369	**0.1357(≈)**	0.1902(+)	0.2042(+)	0.2500(+)	0.2329(+)	0.2728(+)	0.2657(+)
	std	0.0062	0.0096	0.0062	0.0068	0.0157	0.0109	0.0044	0.0193
GLI	mean	**0.0730**	0.0778(≈)	0.0956(+)	0.1365(+)	0.1007(+)	0.1420(+)	0.1661(+)	0.1698(+)
	std	0.0050	0.0030	0.0032	0.0031	0.0096	0.0078	0.0047	0.0070
Leukemia	mean	0.1354	**0.1092(−)**	0.1380(≈)	0.1404(≈)	0.1128(≈)	0.1506(≈)	0.2163(+)	0.1982(+)
	std	0.0021	0.0059	0.0025	0.0025	0.0232	0.0025	0.0065	0.0252
Lung	mean	**0.1631**	0.1658(≈)	0.1847(≈)	0.1839(≈)	0.1776(≈)	0.1956(≈)	0.2140(+)	0.2354(+)
	std	0.0019	0.0046	0.0019	0.0050	0.0090	0.0077	0.0050	0.0126
LungCancer	mean	**0.0508**	0.0679(≈)	0.0615(≈)	0.0809(≈)	0.0723(≈)	0.0790(≈)	0.0982(+)	0.1005(+)
	std	0.0009	0.0029	0.0011	0.0037	0.0095	0.0021	0.0047	0.0099
MLL	mean	**0.1754**	0.2262(+)	0.2494(+)	0.3573(+)	0.3861(+)	0.3085(+)	0.2373(+)	0.3046(+)
	std	0.0058	0.0025	0.0046	0.0014	0.0125	0.0067	0.0032	0.0158
Orlraws10P	mean	**0.1283**	0.1375(≈)	0.1721(+)	0.1870(+)	0.1649(≈)	0.1868(+)	0.2192(+)	0.2162(+)
	std	0.0024	0.0089	0.0031	0.0055	0.0158	0.0024	0.0021	0.0026
Ovarian	mean	0.1416	0.1523(≈)	0.1874(≈)	0.2290(+)	**0.1123(≈)**	0.2184(+)	0.2373(+)	0.2612(+)
	std	0.0028	0.0018	0.0021	0.0030	0.0106	0.0010	0.0124	0.0234
Prostate	mean	**0.1500**	0.1667(≈)	0.1976(≈)	0.2130(+)	0.1881(≈)	0.2269(+)	0.3003(+)	0.2941(+)
	std	0.0050	0.0043	0.0048	0.0049	0.0155	0.0042	0.0043	0.0217
SMKCAN	mean	**0.1791**	0.1943(≈)	0.2397(+)	0.2095(+)	0.2208(+)	0.2246(+)	0.2647(+)	0.2496(+)
	std	0.0059	0.0060	0.0057	0.0132	0.0148	0.0065	0.0038	0.0187
TOX_171	mean	**0.1324**	0.1599(+)	0.1889(+)	0.1599(+)	0.2164(+)	0.2116(+)	0.3082(+)	0.2212(+)
	std	0.0042	0.0025	0.0034	0.0046	0.0096	0.0014	0.00112	0.0124
Avg. fitness		**0.1330**	0.1387	0.1771	0.1848	0.1778	0.1964	0.2335	0.2275

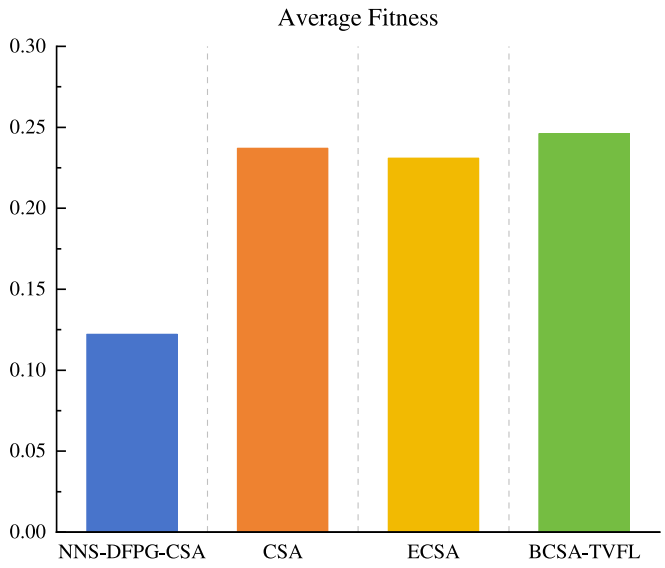


Fig. 11. Average fitness statistic results of the proposed algorithm and the CSA-based algorithm.

In this section, we conducted a detailed comparison of our proposed algorithm with other feature selection algorithms in terms of Feature Selection Rate (All algorithms in this experiment included features from the first stage of filtering when calculating the SFR). From Table 5, our method outperforms other CSA-based algorithms significantly in terms of FSR. However, as shown in Table 6, the proposed algorithm has a slight disadvantage in terms of FSR compared to the ISCA algorithm. Nevertheless, from the above experimental results, it can be seen that the proposed algorithm significantly outperforms ISCA in terms of classification accuracy, which shows that the proposed algorithm in this paper is able to ensure superior classification performance while pursuing efficient feature subset selection. A good balance between feature selection efficiency and classification accuracy is achieved. Figs. 9 and 10 visualize the average FSR statistics for all algorithms using bar charts.

4.4.3. Fitness metric

The fitness metric is an important measure of the comprehensive performance of the feature selection algorithms, taking into account the classification accuracy and feature subset selection rate. Tables 7 and 8 show the performance of all algorithms in terms of fitness. From Table 7, it can be seen that the proposed method significantly outperforms all the CSA-based algorithms. The results in Table 8 further show that the algorithm achieves the best fitness in 10 test problems.

In addition, Figs. 11 and 12 intuitively present the statistical results of the average fitness of all the algorithms. Fig. 11 clearly demonstrates the remarkable success of the proposed algorithm in applying CSA to high-dimensional feature selection problems, while Fig. 12 further validates its effectiveness and competitiveness in feature selection tasks by comparing the results with mainstream heuristics.

4.4.4. Algorithm time consumption comparison

We analyze the computational complexity of NNS-DFPG-CSA for a single iteration. The time complexity is primarily affected by the fitness evaluation, neighbor calculation, and position update steps. The fitness evaluation involves computing the fitness for each crow, which takes $O(D)$ time for each crow. Since there are N crows, the total time complexity for fitness evaluation is $O(N \bullet D)$. The neighbor calculation requires comparing each crow with every other crow, which results in a complexity of $O(N^2 \bullet D)$.

The position update step includes local and global search operations,

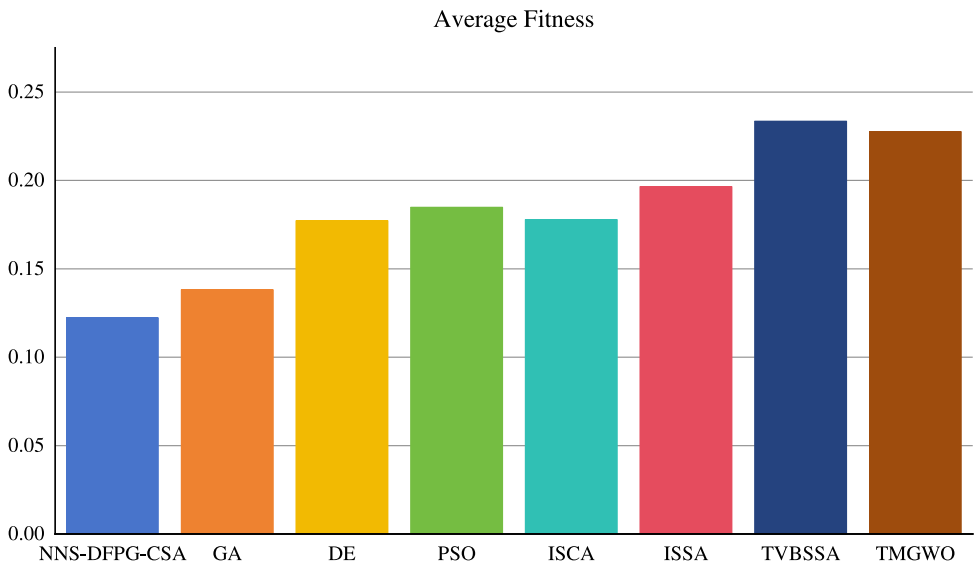


Fig. 12. Average fitness statistics of the proposed algorithm and other compared algorithms.

Table 9
Comparison of NNS-DFPG-CSA and CSA based methods in terms of average and std of time.

Dataset	Stat. measure	NNS-DFPG-CSA	CSA based		
			CSA	ECSA	BCSA-TVFL
ALLAML	mean	275.35	**112.25**	300.25	228.56
	std	3.12	1.35	4.25	0.44
Arcene	mean	260.25	**129.97**	228.99	207.30
	std	2.71	1.96	5.29	0.38
BreastCancer1	mean	371.60	**98.58**	265.09	166.84
	std	2.81	0.24	6.17	0.43
BreastCancer2	mean	347.94	**101.20**	265.56	168.93
	std	2.02	0.15	1.34	0.11
CNS	mean	154.68	**63.60**	123.59	91.76
	std	0.37	0.22	0.73	0.11
CrohnDisease	mean	358.71	**118.12**	296.49	200.38
	std	2.37	0.19	4.70	1.91
GLI	mean	329.15	**88.96**	226.93	149.60
	std	8.22	0.10	4.59	0.64
Leukemia	mean	298.25	**101.36**	221.36	152.78
	std	6.64	0.15	5.56	6.67
Lung	mean	299.68	**145.00**	271.36	271.36
	std	5.15	0.47	7.84	7.84
LungCancer	mean	273.02	**130.14**	247.77	215.44
	std	1.74	0.21	6.45	0.75
MLL	mean	225.36	**118.36**	225.36	142.56
	std	3.89	0.77	6.35	0.24
Orlraws10P	mean	189.56	**109.65**	217.25	198.52
	std	2.36	0.27	2.46	1.58
Ovarian	mean	368.99	**202.68**	338.18	348.54
	std	5.04	1.81	28.93	0.27
Prostate	mean	237.29	**87.16**	182.35	135.35
	std	1.26	0.44	4.10	0.16
SMKCAN	mean	391.12	**165.18**	338.68	306.10
	std	6.26	0.27	12.77	5.60
TOX_171	mean	298.25	**189.25**	255.82	302.72
	std	4.52	0.42	9.28	4.63

each of which takes $O(D)$ time per crow, leading to a total complexity of $O(N \bullet D)$ for all crows. Additionally, after the position updates, the algorithm merges the new positions with the old ones and selects the top N individuals based on fitness, which involves sorting the population. The sorting operation takes $O(N \bullet \log N)$, and selecting the best N individuals takes $O(N)$, which results in a total complexity of $O(N \bullet \log N)$ for this step.

Considering all the steps together, the overall time complexity for a single iteration is dominated by the neighbor calculation, which has a complexity of $O(N^2 \bullet D)$, and the position update step, which contributes $O(N \bullet D + N \bullet \log N)$. Therefore, the total time complexity for one iteration of NNS-DFPG-CSA can be simplified to $O(N^2 \bullet D)$, since $O(N^2 \bullet D)$ dominates the other terms in large-scale problems.

Tables 9 and 10 provide a detailed comparison of time consumption between the NNS-DFPG-CSA algorithm and other algorithms. As observed in Table 9, the average runtime of NNS-DFPG-CSA marginally lags behind that of other algorithms based on CSA. Table 10 presents a comparison of time consumption between NNS-DFPG-CSA and other classic algorithms (DE, GA, PSO), as well as the most recent ones (ISCA, ISSA, TVBSSA, TMGWO). It is evident that TVBSSA consumes the least amount of time among the compared algorithms. However, its classification accuracy and FSR (a measure representing the length of the feature subset of the algorithm) are not optimal. Although NNS-DFPG-CSA incurs a higher time cost compared to the other algorithms, it demonstrates competitive classification accuracy and FSR. Therefore, on a high-performance computer, NNS-DFPG-CSA remains a highly effective feature selection algorithm.

4.4.5. *Effectiveness of the components of NNS-DFPG-CSA*

During the in-depth study of the NNS-DFPG-CSA algorithm, in order to clarify the respective roles of the three improvement strategies proposed in this paper (i.e., dynamic feature probability-guided population initialization strategy, improvement of CSA's local search capability, and improvement of CSA's global search capability) and their contributions to the overall optimization process, the following comparative experiments are designed and implemented in this paper. Specifically, we constructed three variants based on the original NNS-DFPG-CSA algorithm, and each variant removes a key strategy in order to observe its impact on the optimization effect.

- NNS-DFPG-CSA_Variant1: This variant algorithm removes the dynamic feature probability to guide the crow location initialization strategy.
- NNS-DFPG-CSA_Variant2: This variant of the algorithm removes the improvement strategy of this paper for the local search capability of the CSA.
- NNS-DFPG-CSA_Variant3: This variant of the algorithm removes the improvement strategy of this paper for the global search capability of the CSA.

Table 10
Comparison of NNS-DFPG-CSA and other methods in terms of average and std of time.

Datasets	Stat.measure	NNS-DFPG-CSA	Other Method						
			GA	DE	PSO	ISCA	ISSA	TVBSSA	TMGWO
ALLAML	mean	275.35	135.56	125.25	138.23	144.89	125.69	**78.62**	400.69
	std	3.12	0.36	1.42	0.89	2.98	1.22	0.63	3.25
Arcene	mean	260.25	129.48	135.28	136.99	149.11	147.82	**70.81**	385.35
	std	2.71	0.69	1.40	1.55	3.30	0.97	0.24	4.43
BreastCancer1	mean	371.60	98.54	111.12	120.43	138.59	117.43	**69.64**	563.26
	std	2.81	0.44	0.51	0.90	6.15	0.48	0.17	12.09
BreastCancer2	mean	347.94	100.53	113.26	120.42	128.99	119.23	**69.16**	547.59
	std	2.02	0.29	1.46	0.40	11.13	0.45	1.15	3.14
CNS	mean	154.68	63.63	66.90	69.20	69.57	72.50	**38.15**	231.38
	std	0.37	0.36	0.20	0.28	0.32	0.18	0.09	1.05
CrohnDisease	mean	358.71	117.83	129.75	138.03	114.07	138.83	**76.41**	1081.07
	std	2.37	0.67	0.43	0.98	3.44	0.74	0.27	30.88
GLI	mean	329.15	88.65	99.90	108.47	95.88	105.78	**67.76**	788.24
	std	8.22	0.34	0.18	0.47	1.68	0.53	5.75	10.56
Leukemia	mean	298.25	144.25	152.25	138.24	131.02	156.62	**97.28**	780.25
	std	6.64	1.82	0.56	1.82	5.64	0.29	5.23	14.36
Lung	mean	299.68	145.09	151.38	158.26	121.99	166.72	**99.89**	824.58
	std	5.15	1.03	0.38	2.13	1.46	1.11	2.47	35.18
LungCancer	mean	273.02	130.26	136.24	140.59	122.77	150.38	**90.03**	711.18
	std	1.74	0.82	0.39	1.70	5.31	1.22	0.93	29.13
MLL	mean	225.36	142.25	120.54	160.23	117.58	100.25	**89.54**	400.58
	std	3.89	1.56	0.25	2.32	6.25	2.27	4.78	25.63
Orlraws10P	mean	189.56	130.45	127.25	144.72	135.26	107.45	**77.62**	389.72
	std	2.36	1.56	0.82	2.56	4.32	1.25	5.65	26.52
Ovarian	mean	368.99	201.32	209.42	234.31	171.02	233.06	**127.60**	1260.97
	std	5.04	2.09	0.79	25.44	14.14	1.48	10.99	87.18
Prostate	mean	237.29	87.12	93.21	131.87	88.74	100.21	**54.55**	477.03
	std	1.26	0.28	0.30	1.59	1.49	0.33	0.85	8.43
SMKCAN	mean	391.12	165.87	176.24	243.52	129.71	192.30	**98.28**	1274.30
	std	6.26	1.34	0.98	4.40	3.49	1.43	0.19	50.36
TOX_171	mean	298.25	142.96	155.24	230.52	175.62	180.45	**88.62**	855.96
	std	4.52	2.65	0.74	3.22	5.41	0.85	2.36	24.68

Table 11
Statistical results of the average accuracy of NNS-DFPG-CSA with three variants of the algorithm on 16 datasets.

Datasets	Stat. measure	NNS-DFPG-CSA	Variant1	Variant2	Variant3
ALLAML	mean	**0.8627**	0.8527	0.8386	0.8096
Arcene	mean	**0.9178**	0.8956	0.8526	0.8256
BreastCancer1	mean	0.7716	**0.7723**	0.7522	0.7055
BreastCancer2	mean	**0.9610**	0.9425	0.9318	0.8403
CNS	mean	0.7558	**0.8396**	0.8019	0.8105
CrohnDisease	mean	0.8486	**0.8516**	0.8056	0.7535
GLI	mean	0.9195	**0.9218**	0.9035	0.8619
Leukemia	mean	**0.8503**	0.8456	0.8128	0.7926
Lung	mean	**0.8197**	0.8102	0.7992	0.7626
LungCancer	mean	**0.9444**	0.9322	0.9227	0.9105
MLL	mean	0.8057	**0.8089**	0.7526	0.7219
Orlraws10P	mean	**0.8584**	0.8230	0.8038	0.7235
Ovarian	mean	**0.8432**	0.8108	0.8245	0.7962
Prostate	mean	**0.8340**	0.8029	0.7992	0.7692
SMKCAN	mean	0.8017	0.7956	**0.8102**	0.7536
TOX_171	mean	0.8539	**0.8716**	0.8204	0.7435
Avg. Accuracy		**0.8530**	0.8486	0.8270	0.7863

Table 11 lists the statistics of the average classification accuracy of each variant on the 16 high-dimensional datasets. It can be seen that the effects of different variants on classification accuracy are significantly different. First, by comparing variant 1 with NNS-DFPG-CSA, the population initialization strategy using dynamic feature probability guidance can significantly improve the classification accuracy at a fixed number of iterations. This strategy effectively guides the population in a more favorable direction, which speeds up the search process and improves the optimization efficiency to a certain extent. Second, when comparing Variant 2 with NNS-DFPG-CSA, it can be found that the improvement of the local search ability of CSA by the elite neighborhood search can help the algorithm to dig deeper into the subset of features with higher classification accuracy. This improvement enables the algorithm to select the optimal feature combinations more accurately by strengthening the local search capability and avoiding omissions in the feature selection process. Finally, the comparison results between Variant 3 and NNS-DFPG-CSA show that the dynamic feature probability-assisted crow global search strategy can effectively help the algorithm to jump out of the local optimal solution and explore a wider solution space, so as to enhance the global optimization ability of the algorithm and avoid falling into the local optimal dilemma.

In addition, the radar plot in Fig. 13 clearly presents the classification accuracies of the three variants with NNS-DFPG-CSA on 16 datasets. In the figure, the size of the area enclosed by the connecting lines of the accuracy on different datasets reflects the comprehensive performance of each algorithm. It can be seen that NNS-DFPG-CSA shows the best classification results on most of the datasets, and its larger area represents the stability and superiority of the algorithm on multiple datasets, suggesting that NNS-DFPG-CSA is able to achieve more desirable optimization results in a wider range of scenarios. The performance of the other variants, on the other hand, shows different limitations.

4.4.6. *Parameter sensitivity analysis*

This section examines the parameters of NNS-DFPG-CSA. There are three parameters AP^t_{min}, AP^t_{max} , and the rate of change of feature probabilities ξ. The first two are set the same in this paper as in ECSA. We selected a variety of datasets to comprehensively evaluate the extent to which the proposed algorithm is affected by parameter ξ. These datasets include binary classification datasets (BreastCancer2, CNS), a three classifications dataset (CrohnDisease), a four classifications dataset (TOX_171), and datasets with a more complex categorization structure, such as the five classifications Lung dataset and the ten classifications Orlraws10P dataset. Through applications on these datasets, we aim to systematically explore and quantify the specific impact of parameter ξ

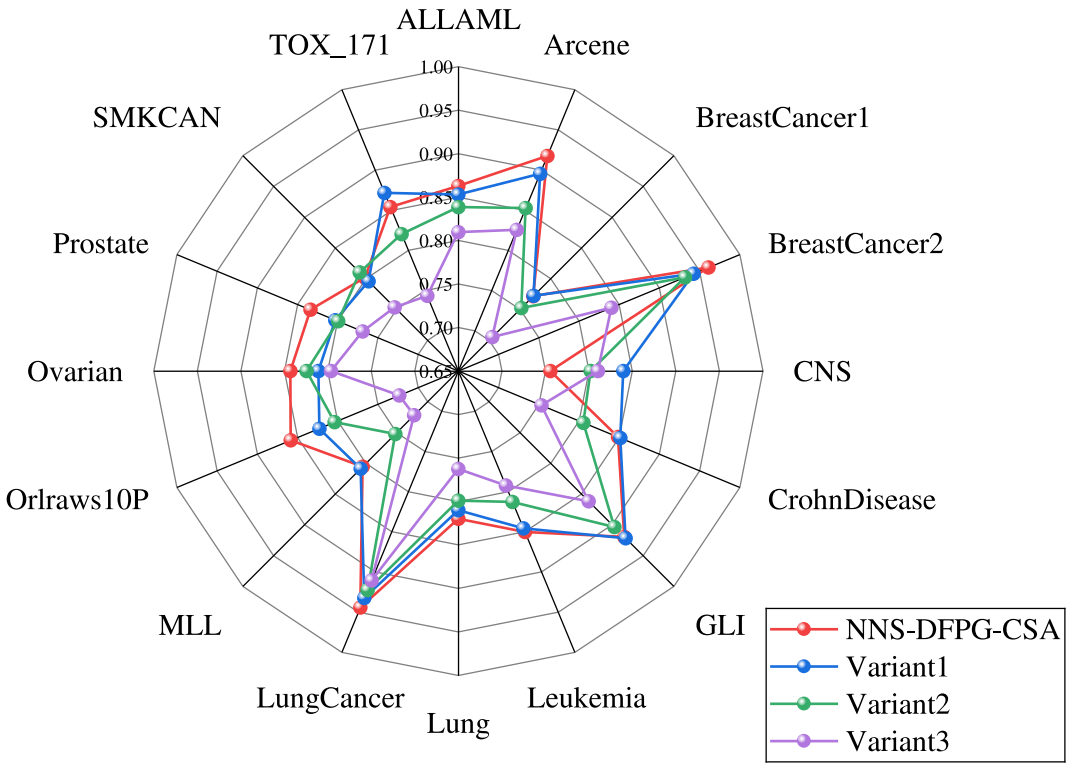


Fig. 13. Visualization of the average accuracy of NNS-DFPG-CSA with each variant of the algorithm.

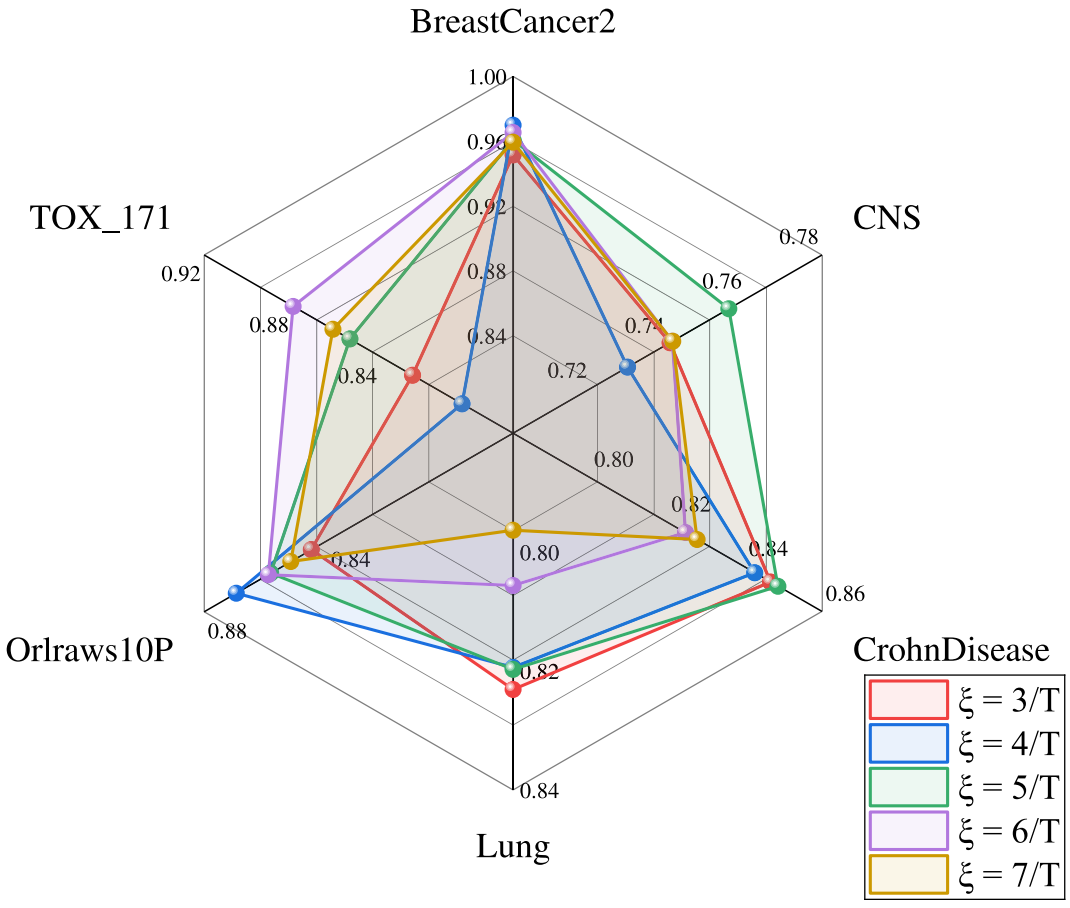


Fig. 14. NNS-DFPG-CSA Parameter Analysis.

on algorithm performance.

Fig. 14 clearly shows the average classification accuracies of the algorithms in the above datasets for different ξ value settings. In the figure, the size of the region enclosed by the accuracy line of each dataset reflects the comprehensive performance of each algorithm. As can be seen from the figure, for most of the datasets, the effect of different ξ settings on the results is more limited, and the performance of the algorithms varies less. However, when A is set to $5/T$, the algorithm shows more stable results. This setting allows the algorithm to maintain a more consistent performance across multiple datasets and avoids performance fluctuations due to large changes in ξ values. Therefore, $\xi = 5/T$ serves as a more appropriate choice to ensure the stability and reliability of the algorithm during the optimization process.

5. Conclusion and future work

In this study, we propose a Crow Search Algorithm with elite neighborhood search and dynamic feature probability guidance for feature selection. The algorithm utilizes TVDAP to balance local and global searches. We employ an innovative elite neighborhood search strategy to enhance the effectiveness of local search. We introduce the concept of dynamic feature probability to generate new solutions, thus the algorithm improves the performance of global search and producing more high-quality solutions. These experiments were conducted on 16 high-dimensional datasets, and our approach was comprehensively compared with CSA-based algorithms and other feature selection methods. The experimental results clearly demonstrate that, among these 16 datasets, our method excels in accuracy and feature selection rate, outperforming most algorithms.

Regarding future work, we plan to further improve the feature scoring component of the algorithm to make dynamic feature probabilities more instructive, thus we can enhance the algorithm's performance. Additionally, combining our method with other feature selection algorithms is an attractive research direction that can further expand the application and performance of feature selection in this field.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Research on this work is partially supported by grants from the Jiangxi Province Science and Technology, PR China (No. 20224BAB202014).

Data availability

Data will be made available on request.

References

Abdel-Basset, M., El-Shahat, D., El-Henawy, I., De Albuquerque, V. H. C., & Mirjalili, S. (2019). A new fusion of grey wolf optimizer algorithm with a two-phase mutation for feature selection. *Expert Systems With Applications, 139*, Article 112824. https://doi.org/10.1016/j.eswa.2019.112824

Anter, A. M., & Ali, M. (2019). Feature selection strategy based on hybrid crow search optimization algorithm integrated with chaos theory and fuzzy c-means algorithm for medical diagnosis problems. *Soft Computing, 24*(3), 1565–1584. https://doi.org/10.1007/s00500-019-03988-3

Askarzadeh, A. (2016). A novel metaheuristic method for solving constrained engineering optimization problems: Crow search algorithm. *Computers & Structures, 169*, 1–12. https://doi.org/10.1016/j.compstruc.2016.03.001

Burczynski, M. E., Peterson, R. L., Twine, N. C., Zuberek, K. A., Brodeur, B. J., Casciotti, L., Maganti, V., Reddy, P. S., Strahs, A., Immermann, F., Spinelli, W., Schwertschlag, U., Slager, A. M., Cotreau, M. M., & Dorner, A. J. (2006). Molecular classification of Crohn's disease and ulcerative colitis patients using transcriptional profiles in peripheral blood mononuclear cells. *Journal of Molecular Diagnostics, 8*(1), 51–61. https://doi.org/10.2353/jmoldx.2006.050079

Chakraborty, C., Kishor, A., & Rodrigues, J. J. (2022). Novel Enhanced-Grey Wolf Optimization hybrid machine learning technique for biomedical data computation. *Computers & Electrical Engineering, 99*, Article 107778. https://doi.org/10.1016/j.compeleceng.2022.107778

Chaudhuri, A., & Sahu, T. P. (2020). Feature selection using Binary Crow Search Algorithm with time varying flight length. *Expert Systems With Applications, 168*, Article 114288. https://doi.org/10.1016/j.eswa.2020.114288

Chowdary, D., Lathrop, J., Skelton, J., Curtin, K., Briggs, T., Zhang, Y., Yu, J., Wang, Y., & Mazumder, A. (2006). Prognostic gene expression signatures can be measured in tissues collected in RNAlater preservative. *Journal of Molecular Diagnostics, 8*(1), 31–39. https://doi.org/10.2353/jmoldx.2006.050056

Chattopadhyay, S., Mishra, S., & Goswami, S. (2016). Feature selection using differential evolution with binary mutation scheme. In *2016 International Conference on Microelectronics, Computing and Communications (MicroCom)* (pp. 1–6). https://doi.org/10.1109/MicroCom.2016.7522533

Cover, T. M., & Thomas, J. A. (2005). *Elements of information theory.*. https://doi.org/10.1002/047174882x

Das, A. K., Das, S., & Ghosh, A. (2017). Ensemble feature selection using bi-objective genetic algorithm. *Knowledge-Based Systems, 123*, 116–127. https://doi.org/10.1016/j.knosys.2017.02.013

Dash, M., & Liu, H. (1997). Feature selection for classification. *Intelligent Data Analysis, 1*(1–4), 131–156. https://doi.org/10.1016/s1088-467x(97)00008-5

Dash, M., & Liu, H. (2003). Consistency-based search in feature selection. *Artificial Intelligence, 151*(1–2), 155–176. https://doi.org/10.1016/s0004-3702(03)00079-1

Dash, R. (2018). An adaptive harmony search approach for gene selection and classification of high dimensional medical data. *Journal of King Saud University - Computer and Information Sciences, 33*(2), 195–207. https://doi.org/10.1016/j.jksuci.2018.02.013

Demšar, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research, 7*(1), 1–30. https://jmlr.csail.mit.edu/papers/volume7/demsar06a/demsar06a.pdf.

Deng, S., Li, Y., Wang, J., Cao, R., & Li, M. (2023). A feature-thresholds guided genetic algorithm based on a multi-objective feature scoring method for high-dimensional feature selection. *Applied Soft Computing, 148*, Article 110765. https://doi.org/10.1016/j.asoc.2023.110765

Deng, X., Li, M., Deng, S., & Wang, L. (2022). Hybrid gene selection approach using XGBoost and multi-objective genetic algorithm for cancer classification. *Medical & Biological Engineering & Computing, 60*(3), 663–681. https://doi.org/10.1007/s11517-021-02476-x

Derrac, J., García, S., Molina, D., & Herrera, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation, 1*(1), 3–18. https://doi.org/10.1016/j.swevo.2011.02.002

Dong, J., Zhong, N., & Ohsuga, S. (1999). Using Rough Sets with Heuristics for Feature Selection. In *Lecture notes in computer science* (pp. 178–187). Doi: 10.1007/978-3-540-48061-7_22.

Elaziz, M. A., Ewees, A. A., Ibrahim, R. A., & Lu, S. (2019). Opposition-based moth-flame optimization improved by differential evolution for feature selection. *Mathematics and Computers in Simulation, 168*, 48–75. https://doi.org/10.1016/j.matcom.2019.06.017

Faris, H., Heidari, A. A., Al-Zoubi, A. M., Mafarja, M., Aljarah, I., Eshtay, M., & Mirjalili, S. (2019). Time-varying hierarchical chains of salps with random weight networks for feature selection. *Expert Systems With Applications, 140*, Article 112898. https://doi.org/10.1016/j.eswa.2019.112898

Faris, H., Mafarja, M. M., Heidari, A. A., Aljarah, I., Al-Zoubi, A. M., Mirjalili, S., & Fujita, H. (2018). An efficient binary Salp Swarm Algorithm with crossover scheme for feature selection problems. *Knowledge-Based Systems, 154*, 43–67. https://doi.org/10.1016/j.knosys.2018.05.009

Gong, W., & Cai, Z. (2013). Differential evolution with Ranking-Based Mutation Operators. *IEEE Transactions on Cybernetics, 43*(6), 2066–2081. https://doi.org/10.1109/tcyb.2013.2239988

Gupta, D., Sundaram, S., Khanna, A., Hassanien, A. E., & De Albuquerque, V. H. C. (2018). Improved diagnosis of Parkinson's disease using optimized crow search algorithm. *Computers & Electrical Engineering, 68*, 412–424. https://doi.org/10.1016/j.compeleceng.2018.04.014

GuyonIsabelle, & ElisseeffAndré. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research. Doi, 10*(5555/944919), Article 944968.

Hancer, E., Xue, B., & Zhang, M. (2017). Differential evolution for filter feature selection based on information theory and feature ranking. *Knowledge-Based Systems, 140*, 103–119. https://doi.org/10.1016/j.knosys.2017.10.028

Hancer, E., Xue, B., Zhang, M., Karaboga, D., & Akay, B. (2017). Pareto front feature selection based on artificial bee colony optimization. *Information Sciences, 422*, 462–479. https://doi.org/10.1016/j.ins.2017.09.028

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. https://dl.acm.org/citation.cfm?id=129194.

Hu, W., Liu, M., Dong, M., Liu, T., Zhang, Y., & Cheng, G. (2023). A greedy-based crow search algorithm for semiconductor final testing scheduling problem. *Computers & Industrial Engineering, 183*, Article 109423. https://doi.org/10.1016/j.cie.2023.109423

Kale, G. A., & Yüzgeç, U. (2021). Advanced strategies on update mechanism of Sine Cosine Optimization Algorithm for feature selection in classification problems. *Engineering Applications of Artificial Intelligence, 107*, Article 104506. https://doi.org/10.1016/j.engappai.2021.104506

Ke, L., Li, M., Wang, L., Deng, S., Ye, J., & Yu, X. (2022). Improved swarm-optimization-based filter-wrapper gene selection from microarray data for gene expression tumor classification. *Pattern Analysis and Applications, 26*(2), 455–472. https://doi.org/10.1007/s10044-022-01117-9

Kira, K., & Rendell, L. A. (1992). The feature selection problem: Traditional methods and a new algorithm. *National Conference on Artificial Intelligence*, 129–134. https://ci.nii.ac.jp/naid/10000073076.

Kononenko, I. (1994). Estimating attributes: Analysis and extensions of RELIEF. In *Lecture notes in computer science* (pp. 171–182). Doi: 10.1007/3-540-57868-4_57.

Li, C., Liang, K., Chen, Y., & Pan, M. (2022). An exploitation-boosted sine cosine algorithm for global optimization. *Engineering Applications of Artificial Intelligence, 117*, Article 105620. https://doi.org/10.1016/j.engappai.2022.105620

Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R. P., Tang, J., & Liu, H. (2017). Feature selection. *ACM Computing Surveys, 50*(6), 1–45. https://doi.org/10.1145/3136625

Li, M., Shang, C., Feng, S., & Fan, J. (2013). Quick attribute reduction in inconsistent decision tables. *Information Sciences, 254*, 155–180. https://doi.org/10.1016/j.ins.2013.08.038

Lin, X., Li, C., Ren, W., Luo, X., & Qi, Y. (2019). A new feature selection method based on symmetrical uncertainty and interaction gain. *Computational Biology and Chemistry, 83*, Article 107149. https://doi.org/10.1016/j.compbiolchem.2019.107149

López, D., Ramírez-Gallego, S., García, S., Xiong, N., & Herrera, F. (2021). BELIEF: A distance-based redundancy-proof feature selection method for Big Data. *Information Sciences, 558*, 124–139. https://doi.org/10.1016/j.ins.2020.12.082

Lu, H., Chen, J., Yan, K., Jin, Q., Xue, Y., & Gao, Z. (2017). A hybrid feature selection algorithm for gene expression data classification. *Neurocomputing, 256*, 56–62. https://doi.org/10.1016/j.neucom.2016.07.080

Mafarja, M., & Sabar, N. R. (2018). Rank based binary particle swarm optimisation for feature selection in classification. In *Proceedings of the 2nd International Conference on Future Networks and Distributed Systems* (pp. 1–6). https://doi.org/10.1145/3231053.3231072

Moradi, P., & Gholampour, M. (2016). A hybrid particle swarm optimization for feature subset selection by integrating a novel local search strategy. *Applied Soft Computing, 43*, 117–130. https://doi.org/10.1016/j.asoc.2016.01.044

Nadimi-Shahraki, M. H., Taghian, S., Mirjalili, S., & Abualigah, L. (2022). Binary Aquila Optimizer for Selecting Effective Features from Medical Data: A COVID-19 Case Study. *Mathematics, 10*(11), 1929. https://doi.org/10.3390/math10111929

Ouadfel, S., & Elaziz, M. A. (2020). Enhanced Crow search algorithm for feature selection. *Expert Systems With Applications, 159*, Article 113572. https://doi.org/10.1016/j.eswa.2020.113572

Peng, N. H., Long, N. F., & Ding, C. (2005). Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 27*(8), 1226–1238. https://doi.org/10.1109/tpami.2005.159

Salesi, S., Cosma, G., & Mavrovouniotis, M. (2021). TAGA: Tabu Asexual Genetic Algorithm embedded in a filter/filter feature selection approach for high-dimensional data. *Information Sciences, 565*, 105–127. https://doi.org/10.1016/j.ins.2021.01.020

Seijo-Pardo, B., Bolón-Canedo, V., & Alonso-Betanzos, A. (2018). On developing an automatic threshold applied to feature selection ensembles. *Information Fusion, 45*, 227–245. https://doi.org/10.1016/j.inffus.2018.02.007

Storn, R., & Price, K. (1997). Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces. *Journal of Global Optimization, 11*(4), 341–359. https://doi.org/10.1023/A:1008202821328

Sindhu, R., Ngadiran, R., Yacob, Y. M., Zahri, N.a. H., & Hariharan, M. (2017). Sine–cosine algorithm for feature selection with elitism strategy and new updating mechanism. *Neural Computing and Applications, 28*(10), 2947–2958. https://doi.org/10.1007/s00521-017-2837-7

Song, X., Zhang, Y., Gong, D., & Gao, X. (2021). A fast hybrid feature selection based on Correlation-Guided clustering and particle swarm optimization for High-Dimensional data. *IEEE Transactions on Cybernetics, 52*(9), 9573–9586. https://doi.org/10.1109/tcyb.2021.3061152

Song, X., Zhang, Y., Gong, D., & Sun, X. (2020). Feature selection using bare-bones particle swarm optimization with mutual information. *Pattern Recognition, 112*, Article 107804. https://doi.org/10.1016/j.patcog.2020.107804

Song, X., Zhang, Y., Guo, Y., Sun, X., & Wang, Y. (2020). Variable-Size Cooperative coevolutionary particle swarm optimization for feature selection on High-Dimensional data. *IEEE Transactions on Evolutionary Computation, 24*(5), 882–895. https://doi.org/10.1109/tevc.2020.2968743

Talbi, E. (2009). *Metaheuristics.*. https://doi.org/10.1002/9780470496916

Taradeh, M., Mafarja, M., Heidari, A. A., Faris, H., Aljarah, I., Mirjalili, S., & Fujita, H. (2019). An evolutionary gravitational search-based feature selection. *Information Sciences, 497*, 219–239. https://doi.org/10.1016/j.ins.2019.05.038

Tran, B., Zhang, M., & Xue, B. (2016). A PSO based hybrid feature selection algorithm for high-dimensional classification. In *2022 IEEE Congress on Evolutionary Computation (CEC)*. https://doi.org/10.1109/cec.2016.7744271

Tubishat, M., Idris, N., Shuib, L., Abushariah, M. A., & Mirjalili, S. (2019). Improved Salp Swarm Algorithm based on opposition based learning and novel local search algorithm for feature selection. *Expert Systems With Applications, 145*, Article 113122. https://doi.org/10.1016/j.eswa.2019.113122

Unler, A., Murat, A., & Chinnam, R. B. (2010). mr2PSO: A maximum relevance minimum redundancy feature selection method based on swarm intelligence for support vector machine classification. *Information Sciences, 181*(20), 4625–4641. https://doi.org/10.1016/j.ins.2010.05.037

Viola, P. A. (1995). *Alignment by maximization of mutual information.*. https://doi.org/10.21236/ada299525

Wang, H., Niu, B., & Tan, L. (2020). Bacterial colony algorithm with adaptive attribute learning strategy for feature selection in classification of customers for personalized recommendation. *Neurocomputing, 452*, 747–755. https://doi.org/10.1016/j.neucom.2020.07.142

Wolpert, D., & Macready, W. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation, 1*(1), 67–82. https://doi.org/10.1109/4235.585893

Yu, L., & Liu, H. (2003). Feature selection for high-dimensional data: A fast correlation-based filter solution. *International Conference on Machine Learning*, 856–863. http://facweb.cs.depaul.edu/research/vc/medix/2013/Documents/icml03.pdf.

Zhu, Z., Ong, Y., & Dash, M. (2007). Markov blanket-embedded genetic algorithm for gene selection. *Pattern Recognition, 40*(11), 3236–3248. https://doi.org/10.1016/j.patcog.2007.02.007